

Ch5_Network Layer

https://blog.csdn.net/weixin_53580595/article/details/129480116

知 无标题

一、Introduction

How does *sending host* -- network layer datagram --> *receiving host* ?

- **Sender side**: Network layer **takes segments from the transport layer**, encapsulates each segment into a datagram (the name of network-layer packet), passes to the link layer.
- **Intermediate routers**: **datagrams are routed through** intermediate nodes (**routers**)
- **Receiver side**: Receives the datagram from lower layer, **extracts segment and passes to the** transport layer.
- **Network layer** protocols in *every* host,
- Router : **provide logical communication between hosts**

- **逻辑通信** 是指基于 **IP 地址** 的数据传输，它抽象了设备之间的通信，使得无论设备的物理位置如何，数据都可以 **根据网络层的逻辑寻址机制传递到目标设备**。
- 路由器通过 **IP 地址** 和路由表来确定数据包的路径，而这些地址和路径是逻辑概念，与物理硬件设备的具体位置无关。
- **物理通信** 由 **物理层** 和 **链路层** 负责，主要涉及电缆、无线信号、网卡（NIC）和 MAC 地址等实际设备和硬件。

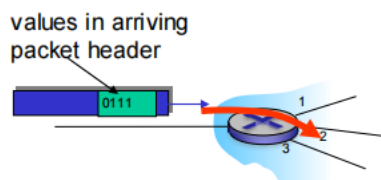
The function of router ?

1. **Forwarding**: move network layer packets from router's **input** to appropriate router **output** – Routers have a **forwarding table**.
2. **Routing**: determine the path of packets as they flow from a **sender** to a **receiver** – Routing algorithm: run at routers to determine the path of packets

由于路由器不直接管理电信号和比特传输的物理过程，它的工作被称为 **逻辑通信**。

Data plane:

- *local*, per-router function
- determines how datagram arriving on router input port is forwarded to router output port



Control plane

- *network-wide* logic
- determines how datagram is routed among routers along end-end path from source host to destination host
- two control-plane approaches:
 - *traditional routing algorithms*: implemented in routers
 - *software-defined networking (SDN)*: implemented in (remote) servers

Forwarding发生在Data plane; **Routing** 发生在Control plane;

1. **Data plane 数据平面**: 数据平面是指路由器负责实际转发数据包的部分，它是路由器工作中直接处理数据传输的模块。
 - 数据平面**主要依据** (**Forwarding Table**) 和相关规则转发数据包。
 - 当数据包到达路由器时，数据平面会基于目标 IP 地址快速查找转发表，决定将数据包发送到哪个输出接口。
 - 此过程发生在路由器的 **硬件层面**，通常通过专用的硬件转发加速（如 ASIC 芯片）实现，以提高效率。
2. **Control plane 控制平面**: 控制平面是负责路由器的路径选择和网络**决策**的部分。
 - 控制平面通过运行路由协议（如 OSPF、BGP 或 RIP）来收集和交换路由信息，构建 (**Routing Table**) 。
 - 根据这些协议，控制平面会选择最佳路径并将其安装到 **数据平面的** (**Forwarding Table**) 中。
 - 此过程通常由路由器的 **软件** 执行，虽然灵活，但速度较慢。

特性	Routing Table	Forwarding Table
维护模块	控制平面 (Control Plane)	数据平面 (Data Plane)
信息类型	包括所有可能的路径	只存储最佳路径
动态更新	随路由协议动态更新	静态导入自路由表，不主动运行协议
用途	用于路径选择 (Path Selection)	用于数据包转发 (Packet Forwarding)
性能	功能强大，但计算速度较慢	精简高效，速度极快
实现方式	软件实现 (路由算法)	硬件加速 (ASIC 或 TCAM 实现)

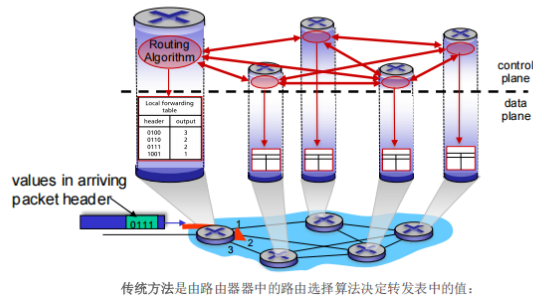
实际工作流程的关系

1. 控制平面通过运行路由协议，填充并维护路由表。
2. 根据路由表中的所有路径，选择最佳路径后将其加载到转发表中。
3. 数据平面利用转发表，快速地对每个数据包做出转发决策。

Two control-plane approaches:

- *Traditional routing algorithms*: implemented in routers

Individual routing algorithm components *in each and every router* interact in the control plane

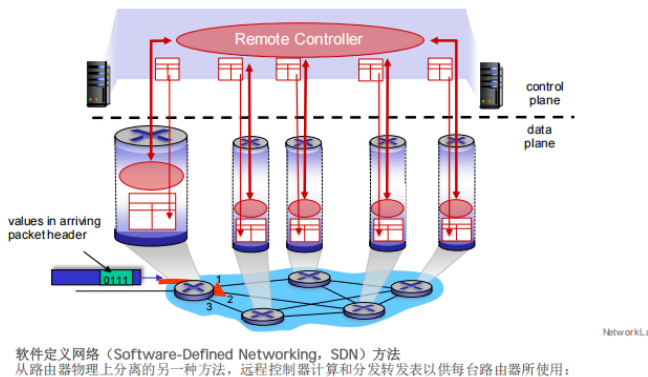


传统网络中，每台路由器或交换机都独立运行控制平面（即算法计算）和数据平面（即数据包转发），每台设备都需要独立运行路由算法，维护自己的路由表。

- 控制平面（Control Plane）和数据平面（Data Plane）紧密耦合在网络设备（如路由器、交换机）中。
- 每台设备独立运行，分别维护路由和转发表。
- 网络设备的功能依赖于专有硬件和固件，灵活性较低。

- *Software-defined networking (SDN)*: implemented in (remote) servers

Remote controller computes, installs forwarding tables in routers



- 在 SDN 中，**控制平面被集中化**，通常通过一个逻辑上集中的控制器（如 SDN 控制器）来管理网络中的所有设备。
 - 这个控制器可能位于云端，也可能部署在本地，但其作用是集中运行路由和策略算法。
 - 数据平面设备（如交换机）简化为只负责接收控制器下发的转发表，并快速进行数据包转发。

Network-layer service model

Internet "**best effort**" service model

因特网的网络层提供了单一的服务，称为**尽力尽为服务（best-effort service）**。

传送的分组既不能保证以它们发送顺序接收，也不能保证它们最终交付；既不能保证端到端时延，也不能保证最小的带宽

Network Architecture	Service Model	Quality of Service (QoS) Guarantees ?			
		Bandwidth	Loss	Order	Timing
Internet	best effort	none	no	no	no

Internet “best effort” service model

No guarantees on:

- successful datagram delivery to destination
- timing or order of delivery 四种delay类型
- bandwidth available to end-end flow

为什么？

All packet share the bandwidth

1. Processing Delay (处理延迟)
2. Queuing Delay (排队延迟)
3. Transmission Delay (传输延迟)
4. Propagation Delay (传播延迟)

其中，Queuing Delay (排队延迟) 是不确定的，因为它取决于网络的实时流量情况。如果路由器的队列很长，排队延迟会增加；如果没有拥堵，排队延迟则可能几乎为零。其他三种延迟通常是比较固定的，受硬件和物理特性影响。

因特网的网络层提供了单一的服务，称为尽力而为服务（best-effort service）。传送的分组既不能保证以它们发送顺序接收，也不能保证它们最终交付；既不能保证端到端时延，也不能保证最小的带宽。

互联网的设计遵循端到端原则（End-to-End Principle）：

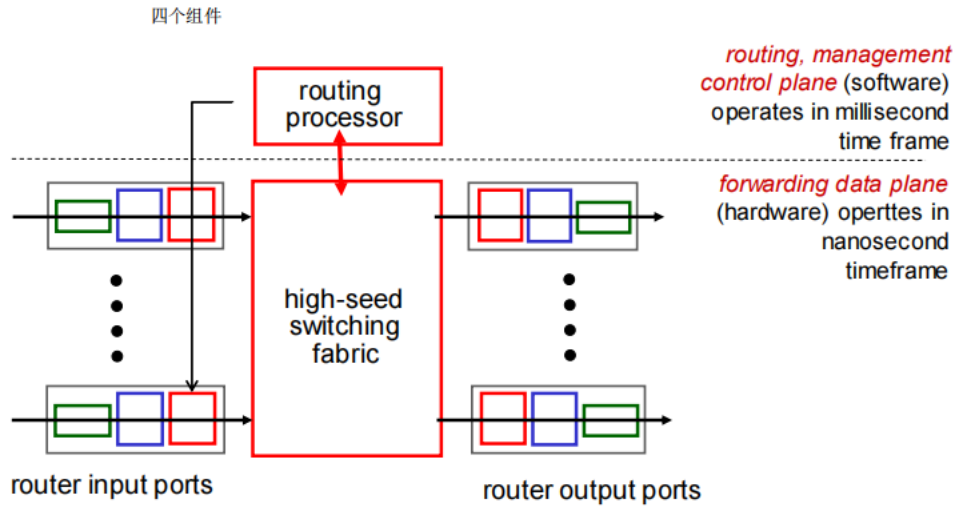
- 网络的核心部分保持简单，复杂的操作（如错误恢复、重传、顺序保证等）由网络边缘的终端设备完成（例如，TCP 协议可以在传输层处理丢包问题）。
- 这种分工进一步降低了核心网络的负担。—— 将复杂性移到边缘

尽力而为模型减少了对资源的占用，使得构建和运行互联网的成本更加可控，同时确保大多数用户能够负担得起。 —— 成本控制

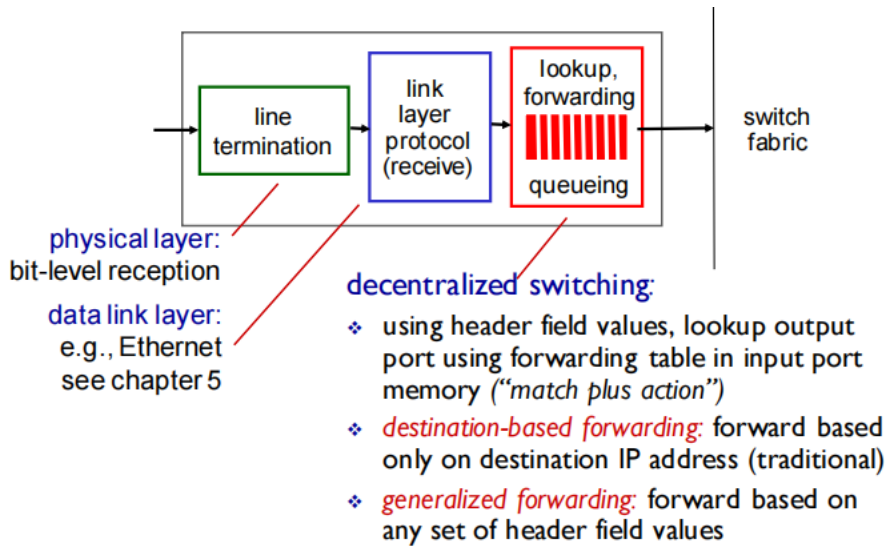
二、What's inside a router

1. Router architecture overview

❖ high-level view of generic router architecture:



1.1. Input port functions



两种转发策略

特点	Destination-based Forwarding	Generalized Forwarding
依据	仅依据目标 IP 地址	任意头部字段（如源地址、端口等）
灵活性	较低	高
实现复杂度	简单，传统硬件即可实现	较复杂，依赖更强大的硬件或软件支持
应用场景	普通网络的简单转发	高级网络控制（如 SDN、QoS、安全）

1. **基于目的地地址的转发** – 这种方式是传统路由器和交换机使用的基本转发方式。数据包的转发仅依据其 目标 IP 地址。

- Destination-based Forwarding 更适合简单的网络环境，强调效率和传统方法。

2. 通用转发 – 工作原理：

数据包的头部字段（如源 IP 地址、目标 IP 地址、协议类型、端口号等）会被解析。

设备使用更复杂的规则（如 ACLs、SDN 控制策略）来决定下一跳或是否丢弃该数据包。

- Generalized Forwarding 则适应现代网络的需求，为实现更多动态、复杂的转发策略提供支持。

Destination-based forwarding 应用

longest prefix match

when looking for forwarding table entry for given destination address, use *longest* address prefix that matches destination address.

在查找给定目的地址的转发表项时，使用与目的地址匹配的最长地址前缀。

1. 找匹配的前缀
2. 找匹配的更多的前缀

Destination Address Range	Link interface
11001000 00010111 00010*** *****	0
11001000 00010111 00011000 *****	1
11001000 00010111 00011*** *****	2
otherwise	3

11001000 00010111 00010110 10100001 which interface?
11001000 00010111 00011000 10101010 which interface?

Decentralized Switching（去中心化交换）是一种网络通信方式，其核心特点是控制和平面操作分布在多个设备上，而不是由一个集中式的控制器完成

传统：

- 性能受限于中央控制器的能力，随着网络规模增长，可能形成瓶颈。
- 当中央设备发生故障时，网络的功能可能完全失效（单点故障）。

去中心化交换：

- 网络更具容错性，因为每个设备都能够独立运行和决策。
- 允许多路径分流，减少拥堵问题，提高整体性能。

交换机/路由器负责 数据的路径选择与转发

1. **Line termination**：线路终止。位于物理层。指网络设备中完成 与物理介质之间的信号接口 的过程。

- a. 它包括接收来自传输介质的信号，并将这些信号转换为适合更高层（如数据链路层）处理的数

字格式。

b. 这一过程通常发生在网络设备（如路由器、交换机）的物理层。

c. 将物理层中的模拟信号或数字信号正确地转化为逻辑数据，以供设备进一步处理。

2. **Link Layer Protocol (receive)：数据链路层**。例如以太网协议（Ethernet Protocol）。它确保 *数据链路的可靠性*，检测并处理数据包中的错误。

3. **Lookup, Forwarding, and Queueing** 查找、转发和排队：

a. 查找与转发：输入端口使用**转发表**（Forwarding Table）在本地存储 中，根据数据包头中的字段（如目的地址）**查找对应的输出端口**。

b. 排队：如果数据包的到达速度比交换结构（Switch Fabric）可以 处理的速度快，就会临时存放在输入端口的队列中。

这三个功能（Line Termination、Link Layer Protocol Lookup/Forwarding/Queueing）涵盖了交换机或路由器在物理层和数据链路层到网络层之间的主要任务。

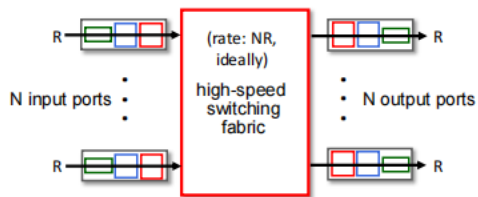
设备类型	层级	依据字段	典型应用场景
二层交换机	数据链路层	MAC 地址	局域网内设备之间的快速通信
三层交换机	网络层	IP 地址	子网间路由或局域网的拓扑管理
多层/应用层交换机	传输层、应用层	TCP/UDP 端口，应用字段	负载均衡、流量优化、大型数据中心或云环境

为何主要限制在低层级？

传统二层交换机的设计目标是简化和加速数据转发，因此只关注低层信息（如 MAC 地址），避免处理更复杂的网络层和传输层信息，减少开销。

1.2. Switching fabrics

- transfer packet from input link to appropriate output link

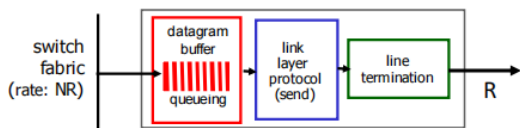


- Switching Fabrics 属于 **数据平面** 的核心功能，它是路由器或交换机内部用于连接输入端口和输出端口的架构。
- 它决定了数据包从某个输入端口转发到对应的输出端口的路径。

工作层级：

- 数据平面任务，与网络层（如转发和路径选择）密切相关。

1.3. Output port queuing



- **Buffering** required when datagrams arrive from fabric faster than link transmission rate.

Datagrams can be lost due to congestion, lack of buffers

1. Switch Fabric:

图中显示了数据从交换结构（Switch Fabric）进入的过程。

- 数据包从交换结构以 **速率 NR** 到达缓冲区。

2. Datagram Buffer:

缓冲区用于临时存储从交换结构发送的分组（Datagram）。

缓冲的目的：

- 当数据包到达的速度（NR）高于链路的发送速率（R）时，缓冲区确保数据不会直接丢失。
- 缓解流量峰值情况下的网络压力。

注意：当缓冲区已满时，如果还有新的数据包到达，则会导致 **数据包丢失**。

3. Link Layer Protocol (Send):

- 链路层负责进一步处理缓冲区中排队的分组。
- 包括链路层协议的错误检测和可靠性功能。

4. Line Termination:

- 在分组通过链路层后，传递到 **Line Termination（线路终止）**，将数据转换为适合物理传输的信号。
- 最终以 **速率 R** 输出数据。

核心概念解析

1. 为什么需要缓冲？

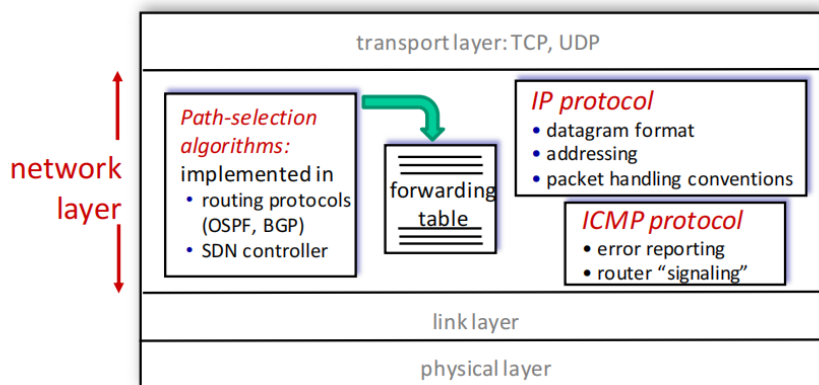
- 当分组从交换结构以更快的速率到达，而输出链路的带宽有限（R 较小）时，需要缓冲来存储这些分组。
- 没有缓冲的话，会导致数据包直接丢弃。

2. 拥塞和丢包现象：

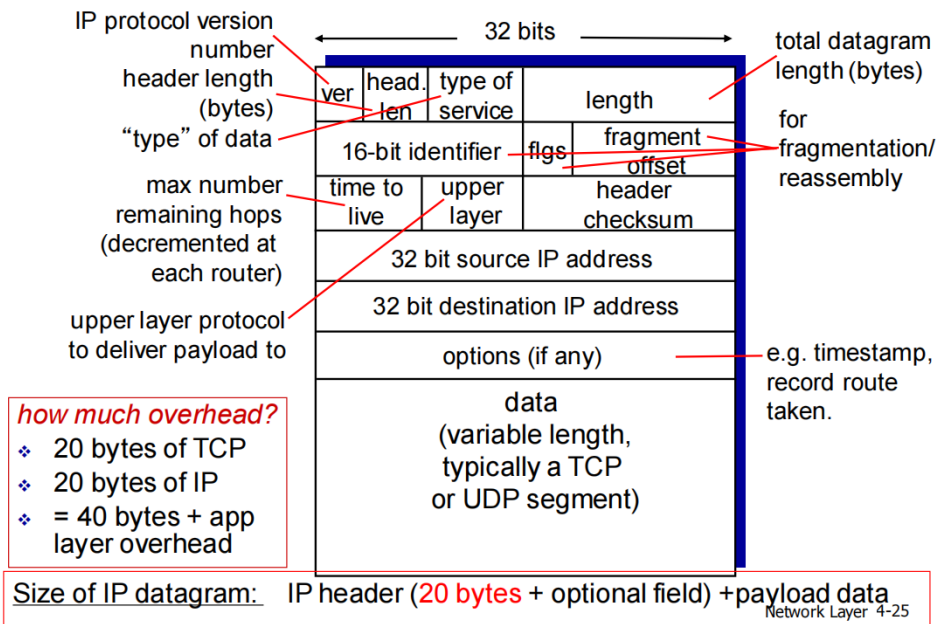
- **拥塞**：当数据包到达速率 NR 长时间高于输出速率 R 时，缓冲区会快速填满，导致后续数据包无法存储。
- **丢包**：由于没有足够的缓冲空间，无法存储新的数据包，这些数据包被丢弃。

三、IP: Internet Protocol

1. datagram format



IP datagram format (IPv4)



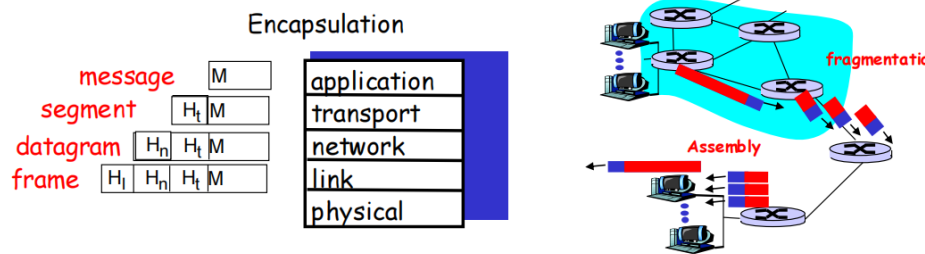
IPv4地址就是给Internet上的每一台主机(或路由器)的每一个接口分配一个在全世界范围内唯一的**32比特bit**的标识符 -- 32bit是地址的长度 ~ 点分十进制

“Overhead” 指的是在传输数据时，为了确保数据能够正确地传递和路由，而在实际数据（也称为“payload”）之外添加的额外数据。例如，在这张图中提到的 TCP 和 IP 数据报：

- TCP头部需要20字节。
- IP头部也需要20字节。
- 合计40字节的“overhead”，再加上应用层可能会有的额外开销。

这些额外的数据是为了标记包的来源、目的地、校验等内容，使得网络协议能够正常运行。但它们会增加总的数据传输量，从而也可能影响传输效率。

IP Datagram and Link-layer Frame



- ❖ IP datagram is encapsulated within the link-layer frame for transmission from one router to the next router
- ❑ **MTU (Maximum Transmission Unit)**: MTU is the size of the largest possible transmission unit. Different link types may have different MTUs
 - MTU of Ethernet is 1500 byte: Ethernet frames can carry up to 1500 bytes for each frame.
 - Some wide-area links have MTU=576 bytes
- ❖ What happens if each of the links along the path between the sender to the destination? ---**Fragmentation and Reassembly**

26

MTU (Maximum Transmission Unit) 是指一个网络链路上能够传输的**最大数据包大小**

MTU 是包含协议**头部**（如 IP 和 TCP 的 header）在内的总数据大小。

1. Fragmentation (分片)

当数据包的大小**超过链路的 MTU**时，它无法直接传输，必须将数据包分成多个片段，以符合链路的 MTU 限制。这通常由发送端的网络协议（例如 IP 协议）来处理。

为了适应 MTU 限制，IP 协议会将原始数据报分成更小的片段（fragments），并为每个片段添加独立的 IP 头部，使它们可以作为独立的数据包传输。

2. Reassembly (重组)

当数据包的各个片段到达目的地时，接收端的**网络协议**会将这些片段**重新组合**，恢复为原始的数据包，以供应用程序使用。

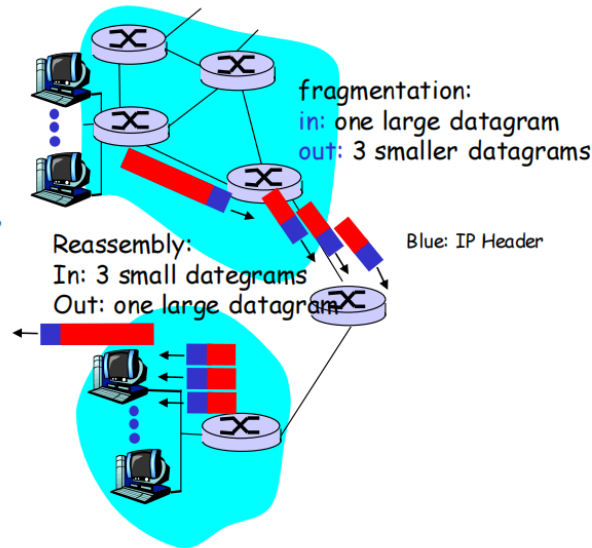
IP Fragmentation & Reassembly

❖ Fragmentation:

- Divide a large IP datagram into two or more smaller IP datagrams;
- Encapsulate each of these smaller IP datagrams in a separate link-layer frames;
- Send these frames over the outgoing link.
- The process is called as fragmentation, and each of these smaller datagrams is referred to as a fragment.

❖ Reassembly: when these fragments arrive at their destination, the destination reassembles these fragments to reconstruct the original larger size datagram.

- Length, Identification, flags, and fragmentation offset fields in the IP header are used to perform the fragment and reassembly task



1. 标识字段 (Identification)

发送端为每个原始数据报分配一个唯一的标识 (Identification) 值，用于标记属于同一数据报的所有片段。

接收端通过检查标识字段，将属于同一数据报的片段聚集起来，准备重组。

2. Flags (标志)

Flags字段决定是否可以对数据报进行分片，以及当前片段是否是最后一个片段。

当Flags字段指示某个片段是最后一个片段时，接收端知道可以完成重组。

3. Fragmentation Offset (分片偏移) 的作用

偏移字段表示当前片段在原始数据报中的位置，以字节为单位。

接收端通过这个偏移量将片段数据正确地排列在重组后的数据报中。

IP Fragmentation and Reassembly

Example

- 4000 byte datagram
- MTU = 1500 bytes

1480-segment的大小

1480 bytes in data field

offset =
1480/8

length	ID	MF flag	offset
=4000	=x	=0	=0

One large datagram becomes several smaller datagrams

length	ID	MF flag	offset
=1500	=x	=1	=0
=1500	=x	=1	=185
=1040	=x	=0	=370

Steps:

1. Subtract 20 from original length: $4000 - 20 = 3980$ (bytes of "IP payload")
2. Subtract 20 from new MTU: $1500 - 20 = 1480$ (max. bytes of data in each fragment)
3. Divide "maximum data bytes" by 8 to get offset increment: $1480/8 = 185$
4. Offset of each fragment "n" ($n = 1, 2, 3, \dots$) = $(n-1) \times$ "offset increment": 0, 185, 370. ...
5. Length of each fragment (except the last fragment) = MTU = 1500 bytes.
Length of last fragment = $20 +$ remaining data bytes = $20 + 3980 - 2 \times 1480 = 1040$.

Note that fragment offset is in unit of 8-byte

28

1. 什么是分片?

当数据报的大小超过网络链路支持的最大传输单元 (MTU) 时, IP协议会将数据报分成更小的片段, 使每个片段都能适配 MTU 的限制。

- 图中的 原始数据报 长度为 4000 字节, 其中 20 字节为 IP 头部, 剩下 3980 字节为有效载荷 (payload)。
- 链路的 MTU 为 1500 字节, 每个片段需要保留 20 字节作为头部, 因此可以容纳的有效数据最大为 $1500 - 20 = 1480$ 字节。

2. 分片的步骤

Step 1: 确定有效载荷大小

从原始数据报中扣除 IP 头部的 20 字节, 得出 3980 字节的有效载荷。

Step 2: 计算每个片段可以包含的数据大小

最大数据大小为 $MTU - 20 = 1480$ 字节。

Step 3: 确定偏移增量 (Offset Increment)

分片偏移量以 8 字节为单位, 因此偏移增量为 $1480 \div 8 = 185$ 。

Step 4: 为每个片段设置偏移量和长度

- 第一个片段: 偏移量 0, 长度为 1500 字节 (20 字节头部 + 1480 字节数据)。
- 第二个片段: 偏移量 185 (1480 字节), 长度为 1500 字节。 ($1480/8=185$)

- 第三个片段：偏移量 370（2960 字节），长度为 1500 字节。
- 最后一个片段：偏移量 555，长度为 1040 字节（20 字节头部 + 剩余数据 1020 字节）。

IP Fragmentation & Reassembly

Advantages of fragmentation

- Fragmentation plays an important role in gluing together the many different link-layer technologies

Disadvantages of fragmentation

- Fragmentation complicates routers and end systems.
- **Security issue**: the attacker sends a stream of small fragments to the target host, none of which has an offset. The target can collapse as it attempts to rebuild datagrams from these packets.

IP version 6 doesn't support fragmentation

优点

- **跨多种链路层技术的连接作用**：分片在不同的链路层（例如以太网、Wi-Fi、卫星通信）之间起到了连接桥梁的作用，即使不同链路的 MTU（最大传输单元）不一致，数据报仍然可以通过分片机制成功传输。

缺点

1. **增加了路由器和终端系统的复杂性**：分片机制需要路由器和接收端处理更多的 IP 头信息以及分片数据，对硬件和软件提出了更高的要求。
2. **潜在的安全问题**：攻击者可能利用分片机制对目标系统进行攻击，例如发送一连串偏移量为零的微小片段，使目标系统在试图重组数据报时发生崩溃。这种攻击被称为 **分片重组攻击** 或 **分片炸弹** (Fragment Bomb) 。

IPv6 的改进

IPv6 不支持分片，因为分片带来的复杂性和安全问题较大。在 IPv6 中：

- 如果数据报的大小超过路径的 MTU，发送端需要在更高的层（如传输层）进行分片，而不是在网络层。

IPv4 与 IPv6

IPv4

版本	首部长度	区分服务	总长度	
标识		标志	片偏移	
TTL	协议	首部校验和		
源地址				
目的地址				
可选字段			填充	

IPv6

版本	通信量类	流标号	
有效载荷长度		下一个首部	跳数限制
源地址			
目的地址			

不变: ■ 更名: ■ 取消: ■ 新增: ■

IPv4

IPv6

单播	
组播	
广播	任播

2. IPv4 addressing

2.1. IP addressing: introduction

- **Interface**: Connection between host/router and [physical link](#)
 - router – multiple → multiple IP address
 - host – one or two
- IP address – each *interface* ~ a [globally unique IP address](#)
- 4-byte IP address is written in **dotted-decimal notation**

223.1.1.1 =

11011111	00000001	00000001	00000001
223	1	1	1

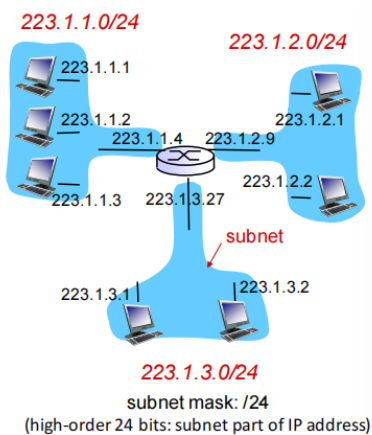
2.2. Subnets

- The structure of *IP address* = **subnet** + *host*
- What's a subnet?
 - A subnet is a network where interfaces can physically reach each other *without passing through an intervening router* – 子网是一个网络，其中设备的接口可以在不通过中间路由器的情况下直接相互通信。
 - 这意味着所有的设备都位于同一个网络区域内，可以直接发送和接收数据，而不需要额外的路由操作。
 - 通常子网用于优化网络性能和管理，更方便地组织和分配网络资源
- IP address of subnet
 - Dotted-decimal notation: a.b.c.d/x, where the notation /x is known as **subnet mask**, it indicate that **x leftmost bits of the 32-bit IP address is the subnet part, and remaining (32-x) bits is the host part**
 - Interfaces in the same subnet have the same subnet part of IP address
 - Example: the above network includes three subnets with subnet mask /24.
 - The subnet mask is used by the TCP/IP protocol to determine whether a host is on the local subnet or on a remote network.

Network Layer 4-32

子网掩码是一种用于网络地址分割的工具，它可以帮助区分一个IP地址中的网络部分和主机部分。简单来说，子网掩码定义了网络的范围，指明哪些部分属于同一个子网。它通常由四组数字构成，类似于IP地址，比如 **255.255.255.0**。

a.b.c.d/x 同一个subnet内的interface有同样的subnet part of IP address



子网是通过将接口从其主机或路由器上分离来创建的。

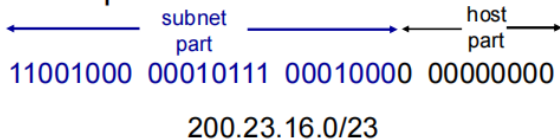
当 IP 地址的节点部分（主机地址）设置为全 0 时，它用于标识子网的网络地址。例如，图片中展示了三个子网：

- **223.1.1.0/24**
- **223.1.2.0/24**
- **223.1.3.0/24**

2.3. IP addressing: CIDR

CIDR: The Internet's address assignment strategy is known as Classless Interdomain Routing (CIDR—pronounced cider)

- subnet portion of address of arbitrary length
- address format: **a.b.c.d/x**, where x is # bits in subnet portion of address



Classless 无阶级的；Interdomain 域间；

无分类域间路由（CIDR，Classless Inter-Domain Routing）

1. CIDR是什么：

CIDR允许子网掩码长度不是固定的，可以是任意长度。

CIDR的地址格式通常写成 **a.b.c.d/x**，其中 **x** 是子网部分的位数。例如，**200.23.16.0/23** 表示该网络地址的前23位是子网部分，后面的位是主机部分。

▼ 与传统划分相比的灵活性：

尽管，CIDR 的 **a.b.c.d/x** 表示法确实和传统子网的表示法在形式上是类似的，都是通过 **/x** 来表示子网部分的位数。

差异：

- 传统的子网划分方式（基于 A 类、B 类和 C 类网络）要求子网掩码有固定的长度，比如 A 类网络默认为 **/8**，B 类网络为 **/16**，C 类网络为 **/24**。
- 而 CIDR 则打破了这种限制，允许任意长度的子网掩码，比如 **/23** 或 **/29**，使得网络划分更精细，资源利用率更高。

2. 图中表示的地址划分：

图中显示了一个IP地址如何被分成子网部分和主机部分。通过二进制分割，前面的连续“1”表示子网部分，而后面的“0”表示主机部分。

For a general interconnected system of routers and hosts, we can use the following recipe to define the subnets in the system:

□ *IP address of subnet*

- When the node part of the IP address is set to all 0s, it is used to identify the network address of a subnet.
- For example, the subnet mask of the left subnet is /24; and its network address is 223.1.1.0
- When the node part of the IP address is set to all 1s, it is a multicast address. That is, this packet is sent to all nodes in this subnet.
- For example, the packet with the destination address 223.1.1.255 will be sent to all nodes in the subnet 223.1.1.0 with network mask /24.

主机部分：

- 全置0 → identify the network address of subnet
- 全置1 → multicast address

子网的网络地址（network address of subnet）是用于标识整个子网的地址。它代表了子网的范围。

1. 如何计算网络地址：

- 网络地址是通过将 IP 地址的主机部分设置为全 0 计算得到的。例如，如果子网掩码是 /24，那么网络地址将包括前 24 位的子网部分，其余部分为全 0。
- 例如，子网 223.1.1.0/24 的网络地址就是 223.1.1.0。

2. 用途：

- 网络地址用于定义子网的范围，例如确定哪些 IP 地址属于同一个子网。
- 它在路由表中用于标记子网位置，帮助路由器更高效地转发数据包。

3. 特殊地址（广播地址）：

- 当主机部分全为 1 时，就会形成子网的广播地址。这种地址用于向子网中的所有设备发送数据包。例如，223.1.1.255 是子网 223.1.1.0/24 的广播地址

2.4. How does a host get IP address?

两种获取 IP 地址的方法：

1. 手动设置（Hard-coded by system admin）

- **Windows:** 通过控制面板（control-panel → network → configuration → tcp/ip → properties）手动配置 IP 地址。
- **UNIX:** 编辑配置文件 /etc/rc.config 以手动指定 IP 地址。

2. 动态获取 (DHCP) :

- 使用 **DHCP 协议**, 设备可以从网络中的 DHCP 服务器动态获取 IP 地址, 无需手动配置。
- **DHCP** – Dynamic Host Configuration Protocol -- dynamically get address from as server

DHCP 的特性:

1. DHCP 实现了“即插即用” (plug-and-play) 的功能, 这意味着设备只需连接到网络, 就可以自动获得所需的 IP 配置。

目标 (Goal) : DHCP 的目标是: 当设备加入网络时, 允许其动态地dynamically从服务器获取 IP 地址, 从而大大简化了网络管理的流程。

▼ Host为什么要获得IP地址, 每个host不都有自己的IP地址吗?

1. IP地址是网络通信的必要条件

IP 地址就像一台主机在网络世界中的身份证号, 用于在网络中识别和定位设备。没有 IP 地址, 主机就无法与其他设备通信, 比如访问互联网、发送数据包等。

2. 动态分配的需求

在很多网络中, 尤其是大型企业或公共网络, 主机的数量可能非常庞大, 并且设备的加入或移除很频繁。如果让每台设备都预设一个固定 IP 地址, 会非常麻烦且难以管理。

因此, 通过 DHCP 等方式动态分配 IP 地址, 可以提高灵活性和效率。

3. 网络资源管理

如果每台主机都预先固定一个 IP 地址, 可能会造成地址资源的浪费。例如, 一些设备可能长期闲置, 但它们的固定 IP 地址却无法分配给其他设备。

动态分配 IP 地址可以让有限的地址资源更高效地被利用。

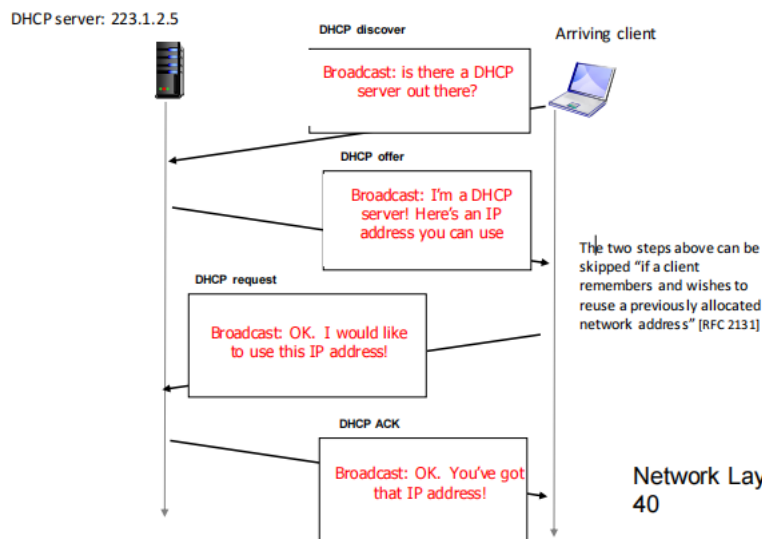
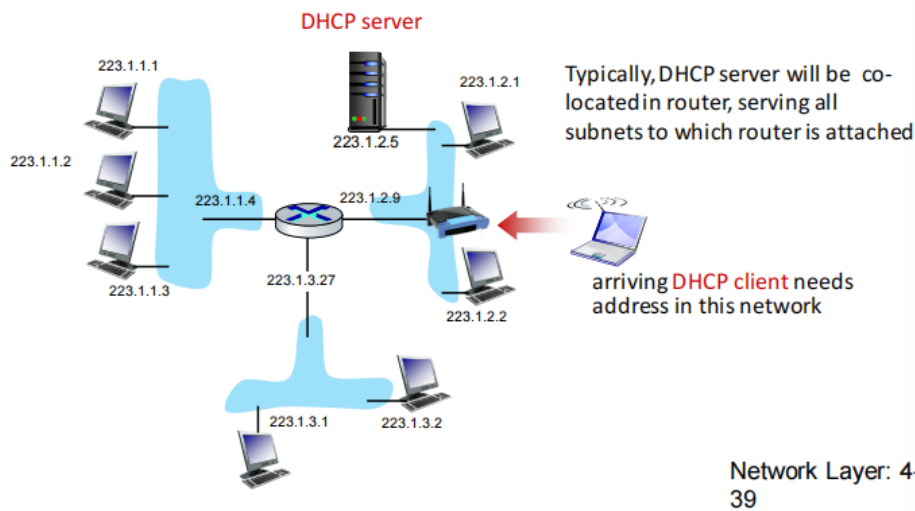
4. 设备移动性和不同网络

当主机移动到不同的网络时 (比如笔记本电脑从家里带到公司), 它原来的 IP 地址可能无法在新网络中使用。因此, 主机需要根据当前网络的情况重新获取一个有效的 IP 地址。

5. 特殊网络配置

在有些场景下, 比如 VPN 或专用网络中, 一个主机可能需要临时获取一个特定范围内的 IP 地址, 以便适应当前网络需求。

DHCP client-server scenario



这张图展示了 DHCP（动态主机配置协议）在客户端与服务器之间的交互过程。以下是具体步骤的说明：

1. DHCP Discover:

- 当新的客户端（主机）加入网络时，会通过广播的形式发送请求，询问网络中是否存在 DHCP 服务器。
- 图中用文字说明：“广播：网络上有 DHCP 服务器吗？”

2. DHCP Offer:

- 如果网络中存在 DHCP 服务器，它会响应客户端的请求，并通过广播的方式发送一个可以使用的 IP 地址。
- 图中用文字说明：“广播：我是 DHCP 服务器！这是一个你可以使用的 IP 地址。”

3. DHCP Request:

- 客户端会再次广播消息，表明它希望使用该 DHCP 服务器提供的 IP 地址。
- 图中用文字说明：“广播：好的，我想使用这个 IP 地址！”

4. DHCP ACK:

- DHCP 服务器会确认客户端的请求，并分配所提供的 IP 地址给它。
- 图中用文字说明：“广播：好的，这个 IP 地址就是你的了！”

此外，图中提到一个补充说明：如果客户端记住了以前分配的网络地址，并希望继续使用它，则可以跳过前两个步骤（DHCP Discover 和 DHCP Offer），这符合 RFC 2131 的规范。

通过这个交互过程，DHCP 实现了“即插即用”（plug-and-play）的网络功能，方便设备在加入网络时自动获取 IP 地址。

▼ 如果刚进入的host没有ip，那么它是如何给别人发送信息的呢？

当一个刚进入网络的主机（host）还没有获得自己的 IP 地址时，它可以使用一种特殊的通信方式来发送信息，这被称为**广播（broadcast）通信**。以下是它的工作原理：

1. 没有IP地址的初始状态:

当主机刚连接到网络时，它还没有分配到自己的 IP 地址，因此无法以常规方式进行通信。

2. 使用广播地址:

主机使用一个特殊的广播地址 `255.255.255.255` 来发送 DHCP Discover 消息。这种地址的特点是可以被网络中的所有设备接收到，而不需要知道它们的具体 IP 地址。

3. 广播的作用:

这种广播通信不要求发送方有一个 IP 地址，因为它的目的是在整个网络范围内寻找 DHCP 服务器。只要网络中有 DHCP 服务器，它就会响应广播消息并提供一个可用的 IP 地址。

4. 低层协议的支持:

在没有 IP 地址的情况下，主机依赖链路层（例如以太网）来传输 DHCP Discover 消息。在链路层中，每个设备都有一个唯一的 MAC 地址，可以作为通信的基础。

5. 获得IP地址后正常通信:

一旦 DHCP 服务器分配了 IP 地址，主机就可以开始以常规方式使用 IP 地址进行通信。

2.5. DHCP-功能多样

DHCP: more than IP addresses

DHCP can return more than just allocated IP address on subnet:

- address of first-hop router for client
- name and IP address of DNS sever
- network mask (indicating network versus host portion of address)

这张图片解释了 DHCP 的功能和它可以提供的额外信息。不只是为主机分配 IP 地址，DHCP 还可以返回一些关键的网络配置参数，例如：

- **第一跳路由器的地址**：主机通过此路由器与外部网络通信。
- **DNS服务器的名称和IP地址**：帮助主机解析域名到 IP 地址。
- **网络掩码**：告诉主机如何区分网络部分和主机部分。

2.6. ISP获取IP地址块？

Q: How does an ISP (or organization) get a block of addresses?

- ❖ The nonprofit organization **ICANN** (the Internet Corporation for Assigned Names and Numbers) manages the IP address space and root DNS servers. It allocates the blocks of IP addresses to regional Internet registries (e.g., CNNIC (China Network Information Center), Network Solutions, CDNCC (Canadian Domain name Consultative Committee), etc.)
- ❖ These regional Internet registries allocate IP addresses to ISPs and organizations in their regions.
- ❖ Once an organization has obtained a block of addresses, it can assign individual IP address to the host and router interfaces in its organization usually using the **DHCP** (Dynamic Host Configuration Protocol)

Network Layer 4-4

1. IP地址空间的管理:

图中提到，IP地址空间由非营利性机构**ICANN（互联网名称与数字地址分配机构）**管理。ICANN负责全世界的IP地址分配以及根DNS服务器的管理。

2. 分配到区域性互联网注册机构:

ICANN会将IP地址块分配给区域性互联网注册机构（如CNNIC，中国互联网络信息中心；Network Solutions；CDNCC，加拿大域名咨询委员会等）。这些区域性注册机构负责管理其所在地区的地址分配。

3. 从区域性注册机构到组织或ISP:

区域性互联网注册机构将IP地址分配给其辖区内的ISP或组织。

4. 组织内部的IP分配:

一旦一个组织获得了IP地址块，它就可以通过像DHCP（动态主机配置协议）这样的工具为内部的主机和路由器接口分配具体的IP地址。

3. ICMP

ICMP: Internet Control Message Protocol

❖ used by hosts & routers to communicate network-level information	<u>Type</u>	<u>Code</u>	<u>description</u>
▪ error reporting: unreachable host, network, port, protocol	0	0	echo reply (ping)
▪ echo request/reply (used by ping)	3	0	dest. network unreachable
	3	1	dest host unreachable
	3	2	dest protocol unreachable
	3	3	dest port unreachable
❖ network-layer “above” IP: ICMP message are carried as IP payload (like UDP or TCP segment).	3	6	dest network unknown
	3	7	dest host unknown
❖ ICMP message: type, code plus first 8 bytes of IP datagram causing error	4	0	source quench (congestion control - not used)
	8	0	echo request (ping)
❖ a node recognizing a transmission problem (TTL exceed, destination unreachable, etc.) generates ICMP messages	9	0	route advertisement
	10	0	router discovery
	11	0	TTL expired
	12	0	bad IP header

ICMP–互联网控制消息协议

ICMP 的作用

1. 网络级信息通信:

- ICMP 是一种用于主机与路由器之间交换network-level 网络级信息的协议。
- 它通常用于报告网络错误 error reporting，例如：目标主机不可达、协议不可达或网络未知等。

2. Ping 请求和回复:

- ICMP 中有一种常见的功能是 回显请求（Echo Request）和 回显回复（Echo Reply），即 Ping。

- 通过 Ping，用户可以测试[目标设备是否可达](#)以及与目标设备之间的网络连通性。

3. 错误报告:

- 当主机或路由器检测到传输问题（如 TTL 超时或 IP 数据报头错误），它会生成 ICMP 消息通知发送方，帮助进行故障排查。

ICMP 消息的结构

- ICMP 消息是以 IP 数据报的有效载荷形式传输的，类似于 UDP 或 TCP。
- 每条 ICMP 消息包含：
 - **类型 (type)**：标识消息的种类。
 - **代码 (code)**：提供消息的具体细分信息。
 - **触发错误的 IP 数据报的前 8 字节**：用于帮助发送方识别问题。

类型	代码	描述
0	0	回显回复 (Ping Reply)
3	0-7	目标网络或主机不可达，协议不可达， 端口不可达，或网络/主机未知
4	0	源端抑制（流量控制，不再使用）
8	0	回显请求 (Ping Request)
9	0	路由通告
10	0	路由器发现
11	0	TTL 超时
12	0	IP 数据报头错误

https://blog.csdn.net/baidu_37964071/article/details/80514340

https://blog.csdn.net/zy_dreamer/article/details/132509931

ICMP:

网络层协议

IP协议不可靠 → 如果丢包了 没有办法通知传输层

需要一种协议完成这样的功能:

1. 确认IP包是否成功到达目的地址
2. 通知在发送过程中IP包被丢弃的原因

需要注意的是：

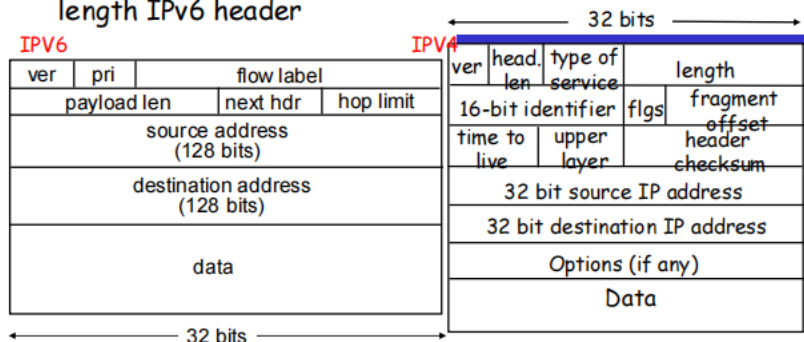
1. ICMP是基于IP协议工作的，但是它并不是传输层的功能，因此仍然把它归为网络层协议
2. ICMP搭配IPv4；IPV6-ICMPv6

ping命令

4. IPv6

4.1. Header

- **Large address space**: 32 bits to 128 bits
- **No fragmentation** in IPv6
- **No checksum field** in order to reduce processing time at each hop
- **No options field** in IPv6, which leads to 40-byte fixed length IPv6 header

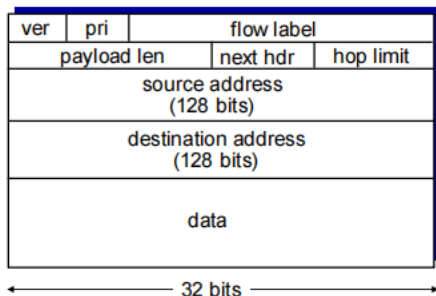


priority: identify priority among datagrams in flow

flow Label: identify datagrams in same “flow.”

(concept of “flow” not well defined).

next header: identify upper layer protocol for data



4.2. Changes from IPv4

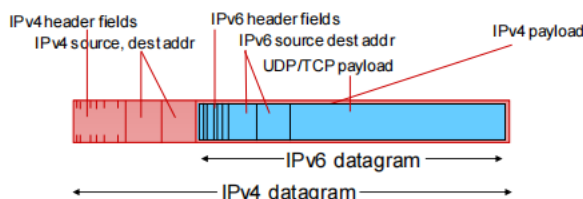
- ❖ **checksum**: removed entirely to reduce processing time at each hop
- ❖ **options**: allowed, but outside of header, indicated by “Next Header” field
- ❖ **ICMPv6**: new version of ICMP
 - additional message types, e.g. “Packet Too Big”
 - multicast group management functions

1. IPv6的新特性:

- 校验和 (Checksum) 移除:
 - IPv6 移除了 IPv4 中的校验和字段, 这样可以减少每一跳中对校验和的处理时间, 提高了路由效率。
- 选项 (Options) 放在扩展头部:
 - 在 IPv4 中, 选项位于基本头部中, 而 IPv6 则将选项字段移到了扩展头部, 并通过“下一个头部”字段 (Next Header) 指示其位置。
- ICMPv6:
 - IPv6 引入了新的 ICMP 版本, 不仅保留了传统功能 (如 Ping 请求和回复), 还增加了新类型消息, 比如“数据包太大” (Packet Too Big) 和组播管理功能。

4.3. Transition from IPv4 to IPv6

- ❖ not all routers can be upgraded simultaneously
 - no “flag days”
 - how will network operate with mixed IPv4 and IPv6 routers?
- ❖ **tunneling**: IPv6 datagram carried as *payload* in IPv4 datagram among IPv4 routers



IPv4到IPv6的过渡机制:

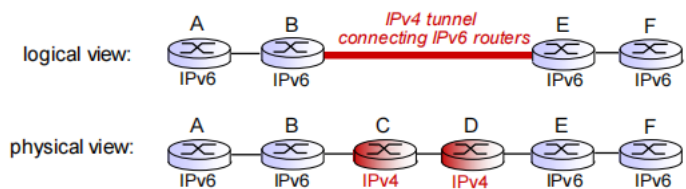
- 逐步过渡:
 - 不可能在所有路由器上同时升级到 IPv6, 因此采用了渐进式的过渡, 无需明确的“旗帜日”。
- 隧道技术 (Tunneling) :

- 在混合网络环境中，通过将 IPv6 数据包作为 IPv4 数据包的有效载荷，在仅支持 IPv4 的路由器间传递，实现 IPv6 数据包的中继。

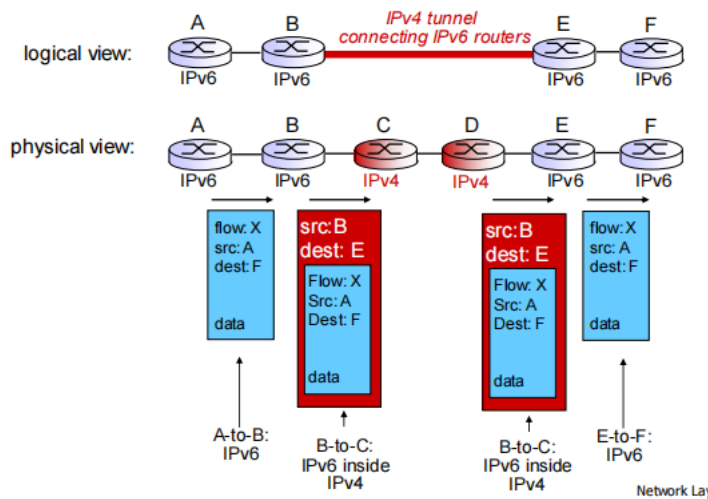
图中显示了一个 IPv4 数据包，其中的有效载荷部分是一个 IPv6 数据包。

IPv4 包含其源地址和目的地址，而 IPv6 包则有独立的源地址和目的地址。

4.4. Tunneling

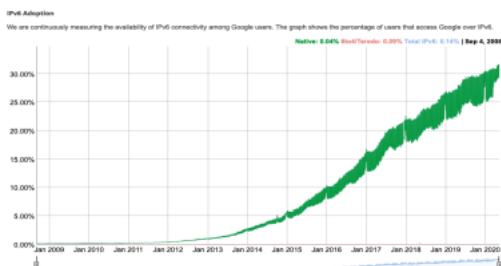


Tunneling



4.5. IPv6: adoption

- ❖ Google¹: ~ 30% of clients access services via IPv6
- ❖ NIST: 1/3 of all US government domains are IPv6 capable



四、Routing algorithms

Network-layer functions:

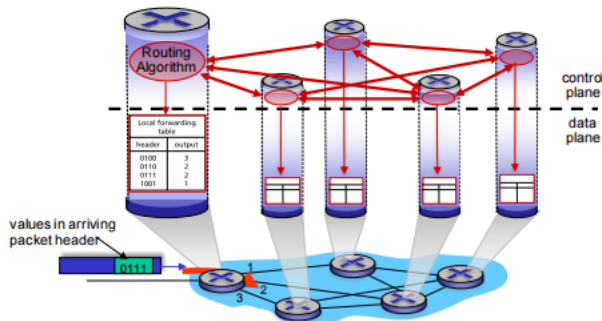
1. forwarding – data plane – inside router – from input to output
2. routing – control plane – macroscopic and holistic – from source to destination

a. Pre-router control (traditional)

预路由控制（传统网络架构）特征：

- 分布式控制：每个设备都具有自己的控制逻辑和转发逻辑，路由决策（如路径选择）在设备本地完成，各设备独立运行。
- 硬件驱动：新功能的实现往往需要更换或升级硬件，灵活性较低。
- 扩展性和管理难度：网络拓扑的变化需要逐步更新每个设备的配置

Individual routing algorithm components *in each and every router* interact in the control plane

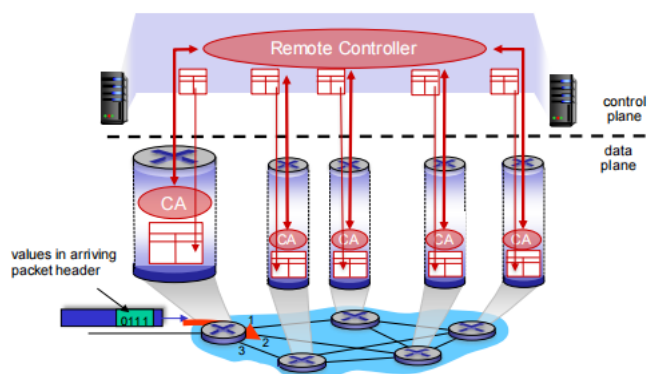


b. Logically centralized control (software defined networking)

逻辑集中控制（软件定义网络，SDN）特征：

- 分离控制平面与转发平面：在SDN中，控制平面（负责路由决策的逻辑）与转发平面（负责数据包转发）分离

Remote controller computes, installs forwarding tables in routers



路由决策 (Routing Decision)

1. 定义:

路由决策是网络设备根据数据包的**目的地址**（通常是 IP 地址），**选择最佳路径将数据包从源点传送到目的地的过程**。

2. 如何工作:

- 网络设备会查看路由表，找到最合适的路径。
- 如果没有明确的路径，可能会选择默认路由或产生错误信息（例如“目标不可达”）。

3. 动态性:

- 路由决策可以是静态的（管理员手动配置路径）或动态的（通过协议如 OSPF、BGP 自动更新路径信息）。

数据包的转发 (Packet Forwarding)

1. 定义:

- 数据包的转发是指网络设备在**收到数据包后**，将其**通过所选路径发送到下一个设备或最终目的地**。

2. 如何工作:

- 转发是基于路由决策结果进行的。设备会根据路由表中的信息，将数据包发送到下一个跃点（例如另一个路由器）。
- 转发逻辑通常位于网络设备的硬件层，速度较快。

Routing protocols

Routing protocol goal: determine "good" paths (equivalently, routes), from sending hosts to receiving host, through network of routers.

- Path: sequence of router packets traverse from given initial source host to final destination host. 路由器数据包从给定的初始源主机到最终目标主机的遍历序列。
- Good: least "cost", "fastest", "least congested"

Graph abstraction

Graph abstraction

graph: $G = (N, E)$

N = set of routers = $\{u, v, w, x, y, z\}$

E = set of links = $\{(u, v), (u, x), (v, x), (v, w), (x, w), (x, y), (w, y), (w, z), (y, z)\}$

Path: a path in a graph $G=(N, E)$ is a sequence of nodes

$(x_1, x_2, \dots, x_p)$ such that each of the consecutive pairs

$(x_1, x_2), (x_2, x_3), \dots, (x_{p-1}, x_p)$ are edges in E .

$c(x, x') = \text{cost of link } (x, x')$

e.g., $c(w, z) = 5$

- if (x, y) does not belong to E (i.e., there doesn't exist link between router x and router y), we set the cost $c(x, y) = \infty$.

- We consider the undirected graphs (i.e., edges do not have a direction). Therefore $c(x, y) = c(y, x)$

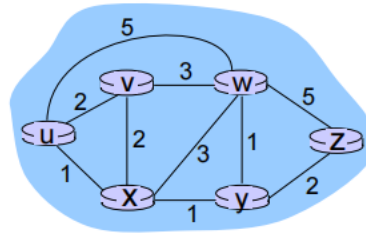
cost of path $(x_1, x_2, x_3, \dots, x_p) = c(x_1, x_2) + c(x_2, x_3) + \dots + c(x_{p-1}, x_p)$

$c(x, z) \neq \text{path}(x, z)$

$c(x, z) = \infty$ 如果没有direct route 那么cost就是无穷

注意! 是cost框定的内容没有直接到的路径

Network Layer 4-65



1. 无向图 $\rightarrow (x, y)$ 和 (y, x) 是一样的, 通信可以双向进行。
2. Path: 从一个源节点到目标节点的路径, 是由一系列节点组成的序列 $(x_1, x_2, \dots, x_p)$ – sequence of nodes
3. cost of link = 路径中每一段边的成本之和 $\rightarrow$ 每条边都有一个成本值, 表示通过该链接传输数据的开销。
4. 如果两个节点之间不存在直接链接 (边不属于 E), 那么它们之间的成本定义为无穷大 (∞) – E 是相邻的set of links

Least-cost path: a path with the least cost.

Least-cost path problem: find the least-cost path between the source router and destination Router.

For example:

Shortest path: The path with the smallest number of links.

Shortest path problem: find the shortest path between the source router and destination router.

If all edges in the graph have the same cost, the least-cost problem is also the shortest path problem.

Network Layer 4-66

1. Least-cost path (最低成本路径)

应用场景: 动态路由协议 (如 OSPF), 可以选择网络中通信效率最高的路径。

成本可以是链路延迟、带宽使用、拥塞程度或其他相关指标。

2. Shortest path (最短路径) – 经过节点和边数最少

不考虑链路成本, 完全基于路径长度。

最短路径问题: 寻找网络中源路由器到目标路由器路径上链接数量最少的路径。

如果网络中的所有边的成本都相同，那么**最低成本路径问题**和**最短路径问题**是等价的，解决其中一个问题即可。

Routing algorithm classification

Routing algorithm classification

Q: global or decentralized information?

global:

每一个node都知道all information

- ❖ all routers have complete topology, link cost info
- ❖ “link state” algorithms

decentralized:

- ❖ router knows physically-connected neighbors, link costs to neighbors
- ❖ iterative process of computation, exchange of info with neighbors
- ❖ “distance vector” algorithms

Q: static or dynamic?

static:

- ❖ routes change slowly over time

dynamic:

- ❖ routes change more quickly
 - periodic update
 - in response to link cost changes

分析路由算法的两种主要维度：**局或分散式信息** 和 **静态或动态特性**

Q: Global / Decentralized:

1. Global:

- 所有路由器都具备完整的网络拓扑信息 complete topology，包括节点之间的链接和链路成本。
- 路由器能够基于完整的网络视图计算出最优路径。

使用“**链路状态 (Link State)**”算法，如 OSPF（开放最短路径优先）

- 使用场景：网络结构相对稳定且需要高效的路径选择。

2. Decentralized:

- 每台路由器仅了解与其直接相连的**邻居信息**和**到邻居的链路成本**。
- 路由器通过与邻居设备交换信息，并进行逐步的计算来更新其路由表。
- 使用“**距离向量 (Distance Vector)**”算法，如 RIP（路由信息协议）
- 适用于较小或动态变化频繁的网络。

Q: Static / Dynamic

1. Static:

- a. 路由表的更新是手动配置完成的，路径变化较慢。
- b. 适用于稳定的小型网络或对变动不敏感的网络环境

又名，**非自适应算法**，不会根据当前测量或者估计的流量和拓扑结构，来调整他们的路由决策。

2. Dynamic:

- a. 路由表能实时更新并适应网络拓扑的变化
- b. 通过周期性更新或响应链路成本变化来调整路径。
- c. 更灵活，但可能带来额外的计算和通信开销
- d. 大型/频繁变动的网络

自适应算法会改变它们的路由决策以便反映出拓扑结构的变化，*通常也会反映出流量的变化情况*。这些动态路由算法在多方面有些差别：获取信息的来源不同（本地、邻居路由器、所有路由器）、改变路径的时间不同（每当拓扑发生变化时、每隔T秒随负载变化）、路由优化的度量不同（距离、跳数、估计的传输时间）

Dijkstra's algorithm

路由算法-链路状态路由_链路状态路由算法-CSDN博客

网络层的主要功能是将数据包从源机器路由到目标机器。在大多数网络中，数据包需要经过多跳才能到达目的地。路由算法和这些算法所用的数据结构是网络层设计的最主要内容。

1. **Global** – 通过 "*link state broadcast*" 实现

链路状态：一种基于向网络中的每个节点发送本地状态更新的路由协议类别。

Link state: i.e., network topology, link costs

- **known to all nodes**, accomplished via link state broadcast
 - msg about a node's neighbors sent to every other node in network
- all nodes have **same global info**

给定一个完整的网络视图，**dijkstra最短路径算法**可以计算出单点到其他所有点的最优路径。

- 在 Dijkstra 算法中，所有节点的网络拓扑和链接成本都是已知的。
- 每次运行时，算法需要利用整个网络的拓扑信息，计算从源节点到目标节点的最短路径。

2. **Dynamic**

- Dijkstra 算法通过逐步更新每个节点到源节点的最短路径，动态地调整路径成本。
- 它根据当前计算的最优路径状态不断更新其它未处理节点的路径成本，因此是动态的。

Dijkstra 算法常用于链路状态路由协议（如 OSPF），计算网络中从某个路由器到其他节点的最低成本路径。

OSPF协议与链路状态算法

- OSPF：开放最短路径优先协议 Open Shortest Path First
- 1. 内部网关协议
- 2. 开放：不受单一厂家限制，公开发表，所有厂家自由使用
- 3. 最短路径优先 – Dijkstra的最短路径优先算法
- 4. OSPF 使用链路状态协议，每个路由器通过 链路状态广播 (Link State Broadcast) 与网络中的所有路由器共享它知道的链路信息。
- 5. 分布式特性
 - 虽然每个路由器都会获取网络的全局拓扑信息，但这些信息是通过 分布式链路状态广播 获取的。
 - 每台路由器只知道与邻居的直接链路状态，然后将这些信息广播给全网。

算法细节

A Link-State Routing Algorithm

Dijkstra's algorithm

- ❖ net topology, link costs known to all nodes
 - accomplished via “link state broadcast”
 - all nodes have same info
- ❖ computes least cost paths from one node (‘source’) to all other nodes
 - gives forwarding table for that node
- ❖ iterative: after k iterations, know least cost path to k dest.’s

notation:

- ❖ $c(x,y)$: link cost from node x to y; $= \infty$ if not direct neighbors
- ❖ $D(v)$: current value of cost of path from source to dest. v
- ❖ $p(v)$: predecessor node along path from source to v
- ❖ N' : set of nodes whose least cost path definitively known

1. 每个节点都了解网络的拓扑结构和链路的成本，所有节点共享相同的信息。
2. 目标：计算从一个节点到其他节点的最小成本。生成对应的转发表（forwarding table）。

Forwarding table v.s. Routing table

- 转发表主要用于实际的数据包转发。当一个数据包到达设备时，设备会通过查询转发表决定下一跳应该将数据包发送到哪里。
- 路由表是网络设备维护的一份完整的路径信息，用于存储到达各个网络目标的最佳路径。
- 转发表从路由表生成并动态更新，转发表从路由表提取最优路径，路由表通过路由协议或静态配置来维护，包含完整的网络路径。

符号定义：

- $C(x,y)$: 从节点 x 到 y 的链路成本。如果两者不是直接相邻节点，则为无穷大 (∞) 。
- $D(v)$: 源节点到目的地节点 v 的当前路径成本值。 – D : Distance
- $p(v)$: 从源节点到 v 的路径中，紧接在 v 之前的前驱节点。
- N : 已明确知道最小成本路径的节点集合。
- 算法是迭代的。经过 k 次迭代后，知道到 k 个目的地的最小成本路径

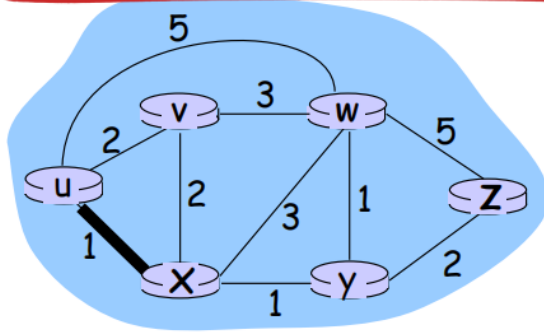
1 **Initialization:**

```
2   $N' = \{u\}$ 
3  for all nodes  $v$ 
4    if  $v$  adjacent to  $u$ 
5      then  $D(v) = c(u,v)$ 
6    else  $D(v) = \infty$ 
7
```

8 **Loop**

```
9  find  $w$  not in  $N'$  such that  $D(w)$  is a minimum
10 add  $w$  to  $N'$ 
11 update  $D(v)$  for all  $v$  adjacent to  $w$  and not in  $N'$  :
12    $D(v) = \min( D(v), D(w) + c(w,v) )$ 
13   /* new cost to  $v$  is either old cost to  $v$  or known
14   shortest path cost to  $w$  plus cost from  $w$  to  $v$  */
15 until all nodes in  $N'$ 
```

Dijkstra's algorithm:



Initialization:

Add the source node into the set N' : $N'=\{u\}$
Initialize the cost other nodes: we classify these nodes into two types:

- (a) For adjacent nodes of u : $D(v)=2$, $D(x)=1$, $D(w)=5$
- (a) For all other nodes: $D(y)=D(z)=\infty$

The 1st iteration:

Compare all nodes without the set N' to find the node with the minimum cost D , that is x , then

- (1) add node x into the set N' . Therefore, $N'=\{u,x\}$.

- (2) the shortest path from source u to x is (u,x) with the cost

$D(x)=1$

- (3) The set of nodes without the set N' is $\overline{N'} = \{v,w,y,z\}$.

Update the cost for the nodes with $\overline{N'}$. We also classify these nodes into two types

- (a) For nodes adjacent to the newly added node x . we use the formula $D(i) = \min(D(i), D(x) + c(x,i))$ to update the cost

$D(v)=\min(D(v), D(x)+c(x,v))=\min(2, 1+2)=2$ through the path (u,v)

$D(w)=\min(D(w), D(x)+c(x,w))=\min(5, 1+3)=4$ through the path (u,x,w)

$D(y)=\min(D(y), D(x)+c(x,y))=\min(\infty, 1+1)=2$ through the path (u,x,y)

- (a) For all other nodes without the set N' and not adjacent to newly added node x , we just copy their previous value.

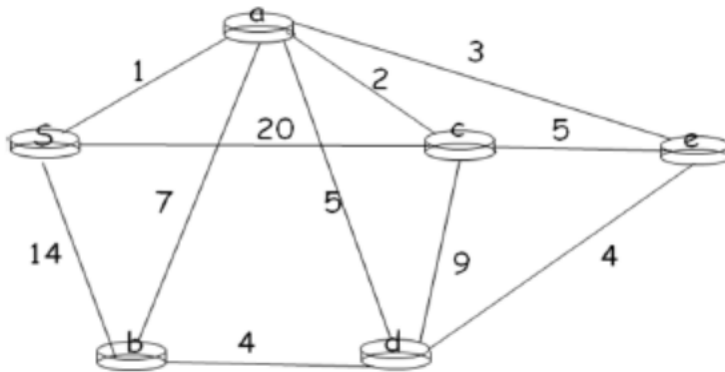
$D(z) = \infty$

Dijkstra 算法是链路状态路由的核心，通过迭代逐步确定最小成本路径，并利用符号对路径信息进行跟踪与优化。

Exercise

The figure below shows the structure of a network.

- Please use Dijkstra's algorithm to find the least-cost paths from the node S to other nodes a , b , c , d and e , respectively.
- Note that you need to give the process of each iteration



The first iteration:

we start with the source node $N'=\{s\}$ --表明N中元素

To get the cost of other nodes:

the adjacent nodes of s: $D(A) = 1; D(B)=14; D(C)=20;$ --写出COST (相邻/不相邻)

all other nodes: $D(D) = \infty; D(e) = \infty;$

Comparing all nodes without the set N' , we find the node with the **minimum cost D** is a, then add node a into the set N' . **We have $N'=\{s,a\}$** , and we find the least cost path from s to a is s-a with the cost 1. --找最短COST, 把那个结点添加到N中

The second iteration:

To get the cost of nodes **without the set N'** :

Update the distance for the adjacent nodes of a: --更新除N中的其他结点的COST

$D(B) = \min[D(B), D(A) + C(A, B)] = \min[14, 1 + 7] = 8$ --找出Min(源路径, 使用刚刚得出的路径+剩余路径)

同理, $D(C)=3; D(D)=6; D(e)=4$

Comparing all nodes without the set N' , we find the node with the **minimum cost D** is c, then **add node c into the set N'** . We have $N'=\{s,a,c\}$, and we find the least cost path from s to c is s-a-c with the cost 3.

The third iteration:

To get the cost of nodes without the set N' :

Update the distance for the adjacent nodes of c: $D(D) = 6; D(e)=4;$ --相邻的找Min

The cost of any other nodes keep unchanged: $D(B) = 8;$ --不相邻的不用管

Comparing all nodes without the set N' , we find the node with the **minimum cost D** is e, then **add node e into the set N'** . We have $N'=\{s,a,c,e\}$, and we find the least cost path from s to e is s–a–e with the cost 4.

The fourth iteration:

To get the cost of nodes without the set N' :

Update the distance for the adjacent nodes of e: $D(D) = 6$;

The cost of any other nodes keep unchanged: $D(B) = 8$;

Comparing all nodes without the set N' , we find the node with the **minimum cost D** is d, then **add node d into the set N'** . We have $N'=\{s,a,c,e,d\}$, and we find the least cost path from s to d is s–a–d with the cost 6.

The fifth iteration:

To get the cost of nodes without the set N' :

update the distance for the adjacent nodes of d: $D(B) = 8$;

Since we only have one node without the set N' ,

we have the least cost path from s to b is s–a–b with the cost 8.

Then we add node b into the set N' . We have $N'=\{s,a,c,e,d,b\}$.

Show the forwarding table:

a) The forwarding table of node S is:

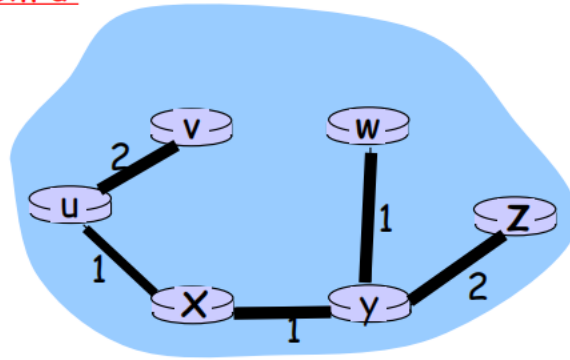
a	(s,a)
b	(s,a)
c	(s,a)
d	(s,a)
e	(s,a)

展示第一跳!

Resulting shortest-path tree from u:

Resulting forwarding table in u:

destination	link
v	(u,v)
x	(u,x)
y	(u,x)
w	(u,x)
z	(u,x)



Distance vector

Distance Vector (DV) Algorithm

DV algorithm is iterative, asynchronous, and distributed.

- **Distributed**: each node receives information from its directly attached neighbors, performs a calculation, and then distributes the results of this calculation back to its neighbors. To find Destination, source node S asks each neighbor X
- **Asynchronous**: it does not require all of the nodes to operate simultaneously.
- **Iterative**: this process continues on until no more information is exchanged between neighbors.

1. Distributed 分布式

在距离向量算法中，每个节点只与直接相连的邻居节点交换信息。

节点会基于邻居传来的信息进行计算并将结果反馈给邻居

2. 异步 (Asynchronous)

距离向量算法不要求网络中所有节点同时更新或同步操作

3. 迭代 (Iterative)

算法需要经过多次迭代，逐步传播和更新路由信息，直至网络中的所有节点达到一致状态（即没有新的信息需要交换时）。

Bellman–Form equation

Bellman-Ford equation (dynamic programming)

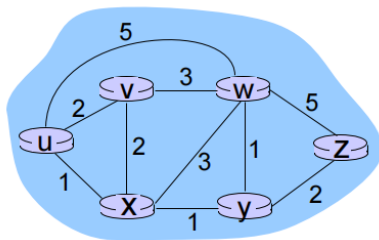
let

$d_x(y) :=$ cost of least-cost path from x to y

then

$$d_x(y) = \min_v \{ c(x,v) + d_v(y) \}$$

$\min$ taken over all neighbors v of x
cost to neighbor v
cost from neighbor v to destination y



clearly, $d_v(z) = 5$, $d_x(z) = 3$, $d_w(z) = 3$

B-F equation says:

$$\begin{aligned} d_u(z) &= \min \{ c(u,v) + d_v(z), \\ &\quad c(u,x) + d_x(z), \\ &\quad c(u,w) + d_w(z) \} \\ &= \min \{ 2 + 5, \\ &\quad 1 + 3, \\ &\quad 5 + 3 \} = 4 \end{aligned}$$

node achieving minimum is next
hop in shortest path, used in forwarding table

Distance vector algorithm

- ❖ $D_x(y)$ = estimate of least cost from x to y
 - x maintains distance vector $D_x = [D_x(y): y \in N]$
- ❖ node x :
 - knows cost to each neighbor v : $c(x,v)$
 - maintains its neighbors' distance vectors. For each neighbor v , x maintains $D_v = [D_v(y): y \in N]$

node存自己的vector和邻居的vector

“距离”：通常表示到达目的地的代价，例如跳数（Hop Count）或其他度量标准。
“向量”：包含目的地和到达目的地的距离信息。

- $D_x(y)$ ：从节点 x 到目标节点 y 的当前最小成本估计值（即最短路径成本）。
- 距离向量 (D_x)：节点 x 维护的所有目标节点 y 的最小成本集合

节点 x 的职责

1. 与邻居的成本 $c(x,v)$:
 - 节点 x 知道到每个邻居 v 的直接链路成本。
 - 例如，若节点 x 和邻居 v 相连，且链路的成本为 5，则 $c(x,v)=5$
2. 邻居的距离向量 (D_v):
 - x 会从每个邻居 v 接收其维护的距离向量 D_v ，并用于更新自身的 D_x

路由算法的基本流程

- 节点定期与直接相邻的路由器交换自己的距离向量。
- 每次接收到邻居 v 的 D_v 后，节点 x 会利用贝尔曼-福特方程更新自己的 D_x

key idea:

- ❖ from time-to-time, each node sends its own distance vector estimate to neighbors
- ❖ when x receives new DV estimate from neighbor, it updates its own DV using B-F equation:

$$D_x(y) \leftarrow \min_v \{c(x,v) + D_v(y)\} \text{ for each node } y \in N$$

iterative, asynchronous:

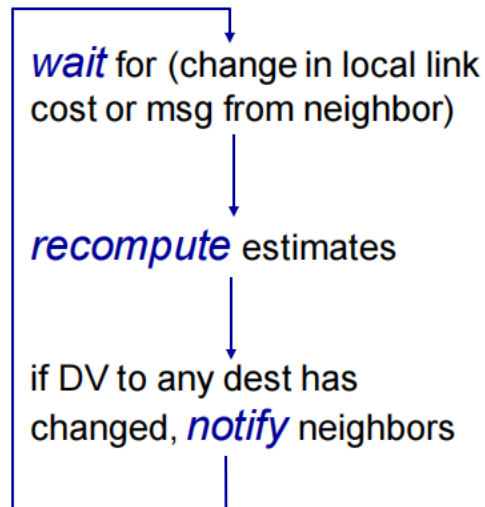
each local iteration
caused by:

- ❖ local link cost change
- ❖ DV update message from neighbor

distributed:

- ❖ each node notifies neighbors *only* when its DV changes
 - neighbors then notify their neighbors if necessary

each node:



1. 等待事件

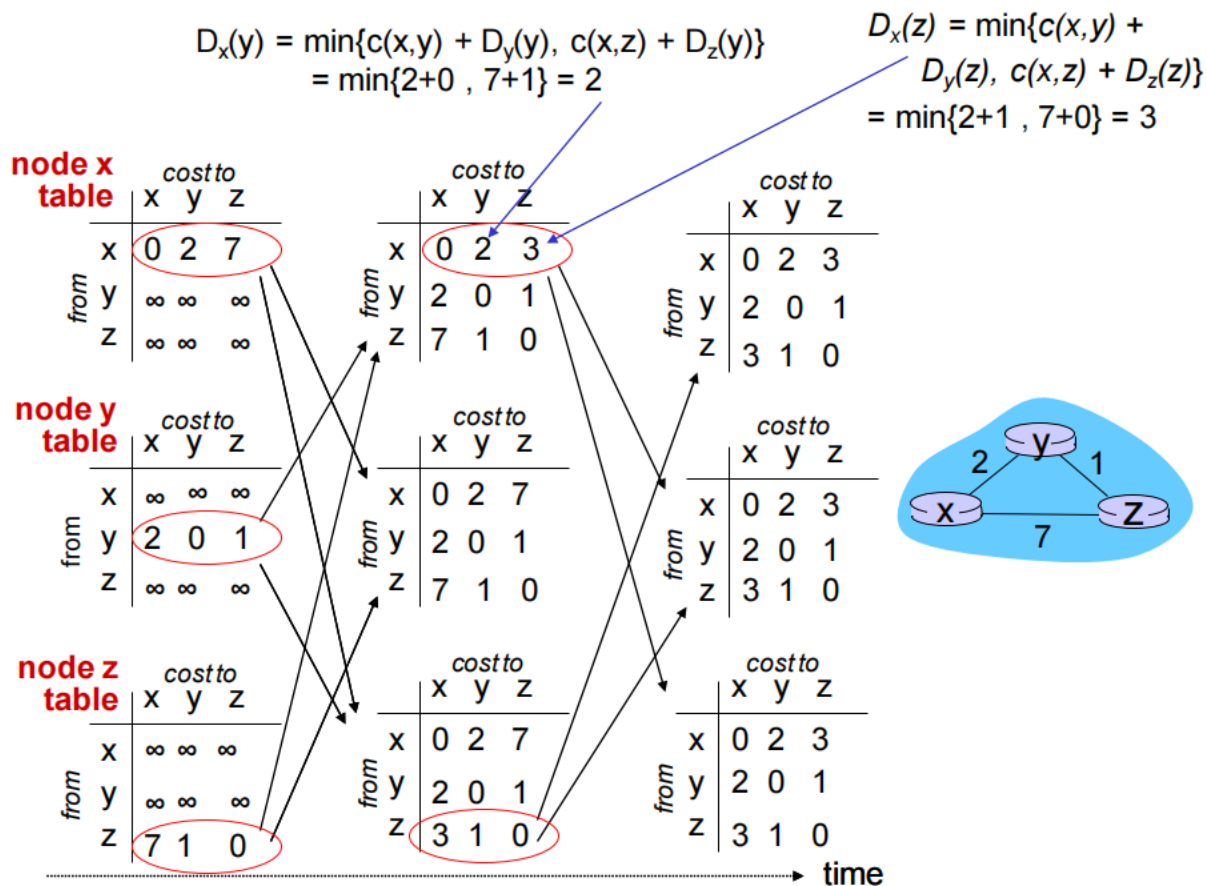
可能是：

- 本地链路成本（如链路延迟或带宽）发生变化。
- 收到邻居发送的距离向量更新消息。

2. 重新计算距离向量

3. 通知邻居

如果计算得出的距离向量 D_x 发生变化，则节点会向所有邻居发送更新消息，让它们根据新信息更新自己的路由。



为什么最终所有节点都会获取一致的最短路径？

这是因为算法的数学基础是贝尔曼-福特方程，它确保了以下特性：

1. 逐步传播最优信息：
 - 每个节点只需与其直接邻居交换信息，但这些邻居会继续把更新信息向其他节点传播，从而实现全网络范围的信息共享。
2. 全局收敛：
 - 算法以迭代的方式逐步优化路径，随着信息的传播，所有节点的距离向量都会更新到最优状态。
 - 一旦没有任何节点再进行更新（即距离向量不再变化），网络就达到了稳定状态（全网一致）。

初始状态：只记录相邻节点。与没有直接连接的节点，路径成本初始化为无穷大，表示不可达。

后来路由信息逐步传播。

特点	距离向量算法	Dijkstra 算法
视图范围	局部视图，依赖邻居信息	全局视图，直接获取整个网络的拓扑结构

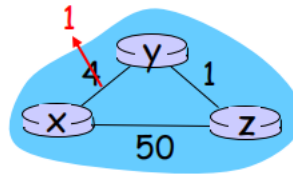
信息传播	逐步扩散	链路状态广播
计算方式	根据邻居的反馈逐步更新	独立计算完整的最短路径
收敛速度	较慢，依赖多次迭代	快速，直接通过全局视图计算
复杂性	简单，计算和存储开销低	复杂，存储和计算全网络拓扑图

- 距离向量算法（如 RIP）通常用于小规模网络，因为其计算和信息交换较为简单。
- Dijkstra 算法（如 OSPF）适用于大型网络，能够快速计算精确的路由。

Distance vector: link cost changes

link cost changes:

- ❖ node detects local link cost change
- ❖ updates routing info, recalculates distance vector
- ❖ if DV changes, notify neighbors



“good
news
travels
fast”

t_0 : y detects link-cost change, updates its DV, informs its neighbors.

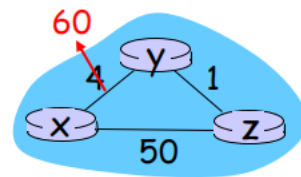
t_1 : z receives update from y, updates its table, computes new least cost to x, sends its neighbors its DV.

t_2 : y receives z's update, updates its distance table. y's least costs do *not* change, so y does *not* send a message to z.

Distance vector: link cost changes

link cost changes:

- ❖ node detects local link cost change
- ❖ *bad news travels slow* - “count to infinity” problem!
- ❖ 44 iterations before algorithm stabilizes: see text



poisoned reverse:

- ❖ If Z routes through Y to get to X:
 - Z tells Y its (Z's) distance to X is infinite (so Y won't route to X via Z)
- ❖ will this completely solve count to infinity problem?

当链路成本发生变化，尤其是变为较大的成本时（例如 60），距离向量算法可能会遇到所谓的“计数到无穷问题”（Count-to-Infinity Problem）

1. 更新过程需要多次迭代，节点逐步传播新的链路成本。
2. 图中提到，这种情况可能需要 **44 次迭代** 才能稳定下来。
3. 在更新期间，错误的路径信息可能被错误传播，导致网络中的节点持续尝试更高成本的路径。

为了解决上述问题，提出了**毒性反转（Poisoned Reverse）**策略：

- 如果节点 ZZ 的最佳路径需要通过邻居节点 YY 到达目标节点 XX：
 - 节点 ZZ 告诉 YY，它到 XX 的距离是无穷大（即毒性反转），这样 YY 就不会选择通过 ZZ 到 XX。
- 毒性反转的作用是阻止路径环路，减少错误路径的传播。

距离向量算法的一个关键缺陷——处理成本变化时可能收敛慢或出现环路问题。解决方案如毒性反转虽然有效，但并不完美，因此距离向量算法通常适合较小规模网络。在更复杂的网络中，链路状态算法（如 OSPF）可能是更好的选择。

Hierarchical routing

Hierarchical routing : Motivation

our routing study thus far - idealization

- ❖ all routers identical
- ❖ network “flat”

... not true in practice

Two key reasons for the use of hierarchical routing
in a large network such as Internet

scale: with billions of destinations:

- ❖ can't store all destination's in routing tables!
- ❖ routing table exchange would swamp links!

administrative autonomy (AS)

- ❖ internet = network of networks
- ❖ each network administrator may want to control routing in its own network

1. 理想化的路由结构

所有路由器是**完全相同**的，并且网络是**扁平化**的，即没有区域或层次划分。

- 然而，这种假设在实际应用中不可能实现，原因是：
 - 网络规模巨大，路由器无法存储所有目的地的路由信息。
 - 扁平网络中的路由表交换会导致链路拥塞，难以管理。

2. 分层路由的需求：两个关键原因

原因 1：规模 (Scale)

- 互联网拥有数十亿的目的地，这使得以下问题变得不可避免：
 - a. **路由表的大小**：在一个扁平化网络中，路由器需要维护到所有目的地的路径，这会导致路由表过于庞大，存储和查询变得困难。
 - b. **路由更新交换的压力**：扁平网络中的路由器频繁交换路由信息可能会占用大量带宽，甚至使链路负载过高。

原因 2：行政自治 (Administrative Autonomy, AS)

- 互联网由多个自治系统 (AS) 组成，每个 AS 是一个独立管理的网络。
- **管理特点**：
 - 每个 AS 的管理员可能希望对自己的网络内的路由进行自主控制。
 - 分层路由允许 AS 内部使用自己的路由策略，而通过边界路由器实现跨 AS 的路由。
- 这种分层设计使得管理员能够更灵活地控制自己的网络，同时减少跨 AS 的复杂性。

- ❖ aggregate routers into regions, “**autonomous systems**” (AS)
 - Each AS consisting of a group of routers that are typically under the same administrative control.
- ❖ routers in same AS run same routing protocol
 - “**intra-AS**” routing protocol: determine the path within the AS.
 - routers in different AS can run different intra-AS routing protocol

gateway router:

- ❖ at “edge” of its own AS
- ❖ has link to router in another AS

aggregate 总数；合计

1. 自治系统（AS）是什么？

- 定义：一个自治系统是由一组路由器组成的网络，这些路由器通常由同一个组织或行政部门控制。
- 目的：AS 的划分旨在简化大规模网络的管理和优化路由选择。
- 特点：
 - 同一个自治系统内部的路由器运行同样的路由协议，称为“**内部路由协议（Intra-AS Routing Protocol）**”。
 - 不同的 AS 可以采用不同的内部路由协议，灵活性更强。

2. 内部路由与外部路由

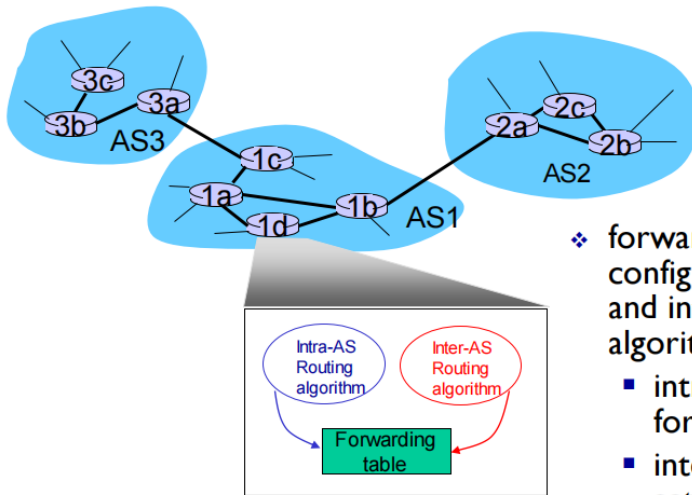
- 内部路由（Intra-AS Routing）：
 - 在 AS 内部，路由器之间使用单一的路由协议，例如 OSPF 或 RIP。
 - 内部路由协议负责在 AS 内寻找最优路径，保证数据包在 AS 内部高效传递。
- 外部路由（Inter-AS Routing）：
 - 如果数据包需要跨越 AS 之间传输，则依赖外部路由协议，例如 BGP（Border Gateway Protocol）。

3. 网关路由器（Gateway Router）

- 定义：网关路由器位于 AS 的边界，负责与其他 AS 中的路由器进行连接。
- 功能：
 - 将内部路由协议生成的信息转化为外部路由协议所需的信息。
 - 通过跨 AS 的链路，将数据包转发到目的地所属的自治系统。

- 意义：网关路由器是自治系统之间通信的关键，它连接了多个 AS，形成互联网的完整路由体系。

Interconnected ASes



- ❖ forwarding table configured by both intra- and inter-AS routing algorithm
 - intra-AS sets entries for internal dests
 - inter-AS & intra-AS sets entries for external dests

外部目的地 (External Destinations) :

- 外部目的地 (例如其他 AS 内的目标) 需要通过内部和外部路由协议共同配置转发表。
- 外部路由协议 (如 BGP) 提供跨 AS 的路径信息，内部路由协议则负责将数据包传递到边界路由器。

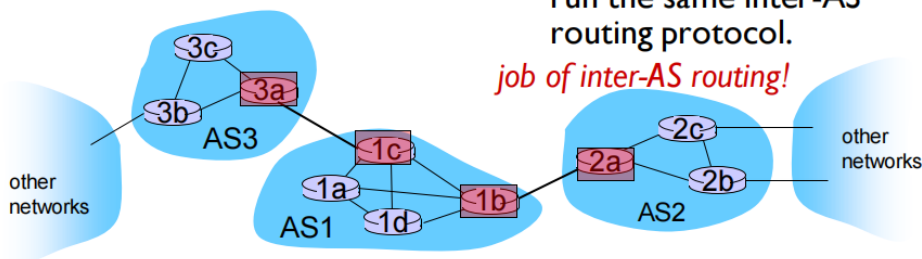
Inter-AS tasks

- ❖ suppose router in AS1 receives datagram destined outside of AS1:
 - router should forward packet to gateway router, but which one?

AS1 must:

1. learn which dests are reachable through AS2, which through AS3
2. propagate this reachability info to all routers within the AS1
3. Neighboring ASs should run the same inter-AS routing protocol.

job of inter-AS routing!



Network Layer 4-94

为了实现跨 AS 的高效数据转发，AS1 必须完成以下任务：

1. 学习目的地的可达性：

- AS1 需要了解哪些目标网络可以通过 AS2 到达，哪些可以通过 AS3 到达。
- 这种可达性信息由外部路由协议（如 BGP，边界网关协议）提供。

2. 传播可达性信息：

- AS1 的网关路由器需要将从 AS2 和 AS3 获取的目标可达性信息传播到 AS1 内部的所有路由器。
- 内部路由协议（如 OSPF 或 RIP）负责分发这些信息，使得 AS1 的所有路由器知道如何将数据包引导到正确的网关路由器。

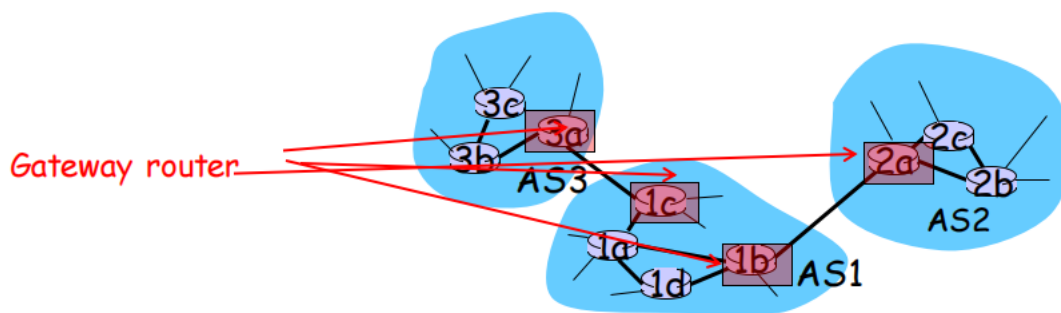
3. 协调路由协议：

- 邻近的自治系统之间需要运行相同的跨 AS 路由协议（通常是 BGP），以确保它们能够顺利交换路由信息。
- 协调协议的目的是实现无缝连接，确保数据包能够跨越多个自治系统到达最终目标。

Gateway router

□ Gateway router

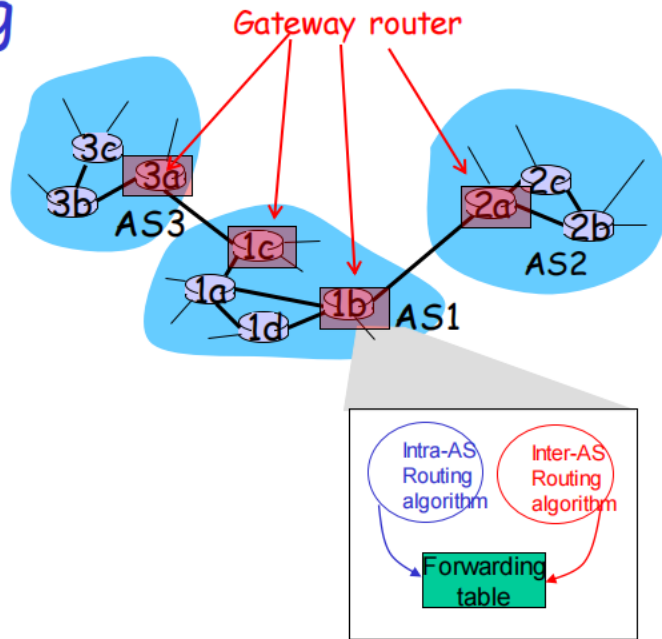
- In each AS, one or several routers take responsibility for forwarding packets to destination outside the AS, which is called *Gateway router*.
- It provides direct link to router in another AS



Hierarchical Routing

□ Gateway router

- Run inter-AS protocol to get the reachability information about the destinations that are reachable through neighboring AS(s).
- Propagate this information to all routers within the AS
- In the Internet, all ASes run the same inter-AS routing protocol, called BGP4 (Border Gateway Protocol).
- configure its forwarding table based on the obtained information from both inter-AS routing protocol and intra-AS routing protocol.
 - Intra-AS sets entries for internal destinations
 - Inter-AS & Intra-As sets entries for external destinations



96

1. 网关路由器在自治系统中的作用

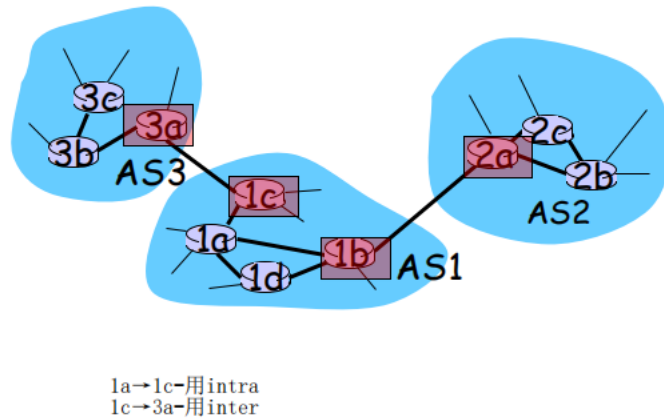
- 运行跨AS路由协议：
 - 网关路由器运行跨AS路由协议（Inter-AS Routing Protocol），如 BGP4（边界网关协议），以获取跨AS的目标可达性信息。
 - 这些信息包括通过邻近AS到达外部网络的可能路径及相关成本。
- 传播可达性信息到整个AS内部：
 - 从跨AS协议获取的信息会被网关路由器转化，并传播到AS内的所有路由器，确保整个自治系统中的路由器都知道如何访问外部目的地。

2. 转发表的配置

- 转发表生成机制：
 - 转发表由两种路由协议协作生成：
 - i. 内部路由协议（Intra-AS Routing Protocol）：负责配置到AS内部目的地的条目。
 - ii. 跨AS路由协议与内部路由协议共同作用：为外部目的地（跨AS的目的地）生成条目。
- 作用：
 - 转发表确保数据包能够正确地从内部目的地转发到外部目的地。
 - 它通过协调内部路由协议和外部路由协议的信息，提供完整的路由路径。

□ Each other router in an AS

- Run an **intra-AS** routing protocol to obtain the information on how to handle the packets to a destination **within this AS**.
- Get the reachability information **from gateway router(s)** about the destinations that are reachable through neighboring **AS(s)**.
- **configure its forwarding table** based on the obtained information from both **intra-AS routing protocol and gateway router(s)**.
 - Intra-AS routing protocol is used to set entries for internal destinations
 - Both Intra-AS and information from gateway router(s) are used to set entries for external destinations



内部路由协议（如 OSPF 或 RIP）用于配置自治系统（AS）内部目的地的转发表条目。

对于外部目的地（位于其他 AS 中的目标），需要结合内部路由协议与网关路由器提供的外部路由信息来配置转发表。

例如，从路由器 1c 到 3a（位于 AS3），路径需要先依赖内部路由协议将数据包送到网关路由器（如 AS1 的边界路由器），再通过网关和跨AS的外部路由（如 BGP）完成传递。

五、Routing in the Internet

Routing in the Internet

Internet **intra**-AS routing: **IGP** (Interior Gateway Protocol)

- Used for routing **within a single Autonomous System (AS)**, like a company's internal network or an ISP's managed network.

Internet **inter**-AS routing: **BGP** (Border Gateway Protocol)

- Between **different** Autonomous Systems on the global Internet.

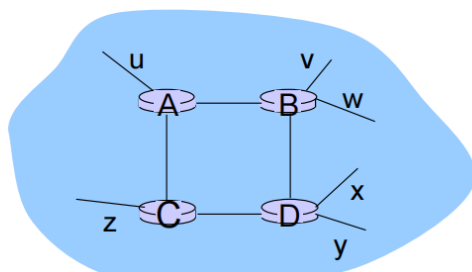
Intra-AS Routing

- ❑ Intra-AS routing in the Internet is also known as **Interior Gateway Protocol (IGP)** since it determines how routing is performed within an autonomous system (AS).
- ❑ Most common Intra-AS routing protocols:
 - **RIP: Routing Information Protocol [RFC 1723]**
 - classic DV: DVs exchanged every 30 secs
 - no longer widely used
 - **EIGRP: Enhanced Interior Gateway Routing Protocol**
 - DV based
 - formerly Cisco-proprietary for decades (became open in 2013 [RFC 7868])
 - **OSPF: Open Shortest Path First [RFC 2328]**
 - link-state routing
 - IS-IS protocol (ISO standard, not RFC standard) essentially same as OSPF

RIP (Routing Information Protocol)

❖ distance vector algorithm

- distance metric: # hops (max = 15 hops), each link has cost 1
- DVs exchanged with neighbors every 30 sec in response message (aka **advertisement**)
- each advertisement: list of up to 25 destination **subnets** (in IP addressing sense)



from router A to destination **subnets**:

subnet	hops
u	1
v	2
w	2
x	3
y	3
z	2

距离向量算法 (Distance Vector Algorithm)

距离度量:

- 距离度量采用**跳数 (hops)** 来衡量, 从一个节点到目的地经过的路由器数量。
- 跳数的最大值为 15 (表明网络规模有限, 再大可能会被认为不可达)。

距离向量交换:

- 每个路由器通过响应报文 (advertisements) 与相邻的路由器交换自己的距离向量表。
- 报文每 30 秒发送一次, 其中包含最多 25 个目的子网的信息。

OSPF (Open Shortest Path First)

- ❖ “open”: publicly available
- ❖ uses link state algorithm
 - LS packet dissemination
 - topology map at each node
 - route computation using Dijkstra's algorithm
- ❖ OSPF advertisement carries one entry per neighbor
- ❖ advertisements flooded to *entire* AS
 - carried in OSPF messages directly over IP (rather than TCP or UDP)
- ❖ *IS-IS routing* protocol: nearly identical to OSPF

OSPF (Open Shortest Path First) 路由协议 是一种常用于内部网（如企业或运营商网络）的动态路由协议。

1. OPEN: 是一种常用于内部网（如企业或运营商网络）的动态路由协议。
2. 使用链路状态算法
 - a. OSPF 基于链路状态路由算法，通过链路状态包（LS Packet）传播网络信息。
 - b. 每个节点维护整个网络的拓扑图，并使用 Dijkstra 算法 计算最短路径。

-- 全局性 => 在使用 Dijkstra 算法的 OSPF 路由协议中，每个节点会通过链路状态广播（Link-State Advertisements, LSAs）获取整个网络的拓扑结构。这种广播允许每个节点拥有全局的网络视图，包括所有节点、链路的状态以及链路的成本。

3. 广播广告信息：
 - a. 广告信息会通过泛洪的方式传播到整个自治系统（AS），确保所有节点拥有统一的网络视图。
 - b. OSPF 消息直接通过 IP 承载，而不是依赖 TCP 或 UDP。
4. 类似协议：IS-IS：
 - IS-IS 路由协议与 OSPF 的实现方式非常相似，广泛用于较大的网络。

- ❑ “open”: protocol specification is publicly available
- ❑ OSPF is hierarchical structure
 - Internal routers;
 - Area border routers
 - Backbone routers
 - Boundary routers
- ❑ Backbone area
 - One area in the AS is configured to be the backbone area.
 - The primary role of the backbone area is to route traffic between different areas in an AS: packets are first routed to an area border router, then routed through the backbone area to the area border router that is in the destination area, and then routed to the final destination.

1. 分层结构:

OSPF 使用分层的路由架构，使其适应大型网络。

- 内部路由器：负责在一个区域内路由。
- 区域边界路由器：连接不同区域。
- 骨干路由器：属于骨干区域，负责区域间路由。
- 边界路由器：连接 OSPF 网络与外部网络。

2. 骨干区域:

在一个自治系统（AS）中，配置一个特殊的区域称为骨干区域。

骨干区域的主要作用是在不同区域之间路由流量：

- 数据包先路由到区域边界路由器。
- 然后通过骨干区域传递到目标区域的边界路由器

骨干区域（Area 0）是什么？

在 OSPF 路由协议中，网络被划分为多个区域（Areas），以减少路由器的计算和存储压力，同时提高网络的规模化能力。骨干区域（Area 0）是一个特殊的区域：

- 它连接所有其他区域，起到“中心枢纽”的作用。
- 每个区域必须直接连接到骨干区域，或者通过虚拟链路间接连接到骨干区域。

<https://cloud.tencent.com/developer/article/2169383>

OSPF (Open Shortest Path First)

- ❑ OSPF is based on the Link State (LS), but more complex
 - each AS may be **divided into multiple areas**
 - Each area uses link-state routing such as Dijkstra's algorithm, with all routers being equal.
 - Within each area, at least one router is connected to the backbone area. The router(s) is called **area border router(s)** and responsible for routing packets outside the area.

1. 基于链路状态协议：

OSPF 使用链路状态路由协议，每个路由器维护整个区域的网络拓扑图。

每个区域内部通过 **Dijkstra 算法** 计算最短路径。

2. 区域划分 (Areas)：

OSPF 网络可划分为多个区域，每个区域中的路由器地位是平等的。

区域划分的目的在于简化网络拓扑，提高网络的扩展能力。

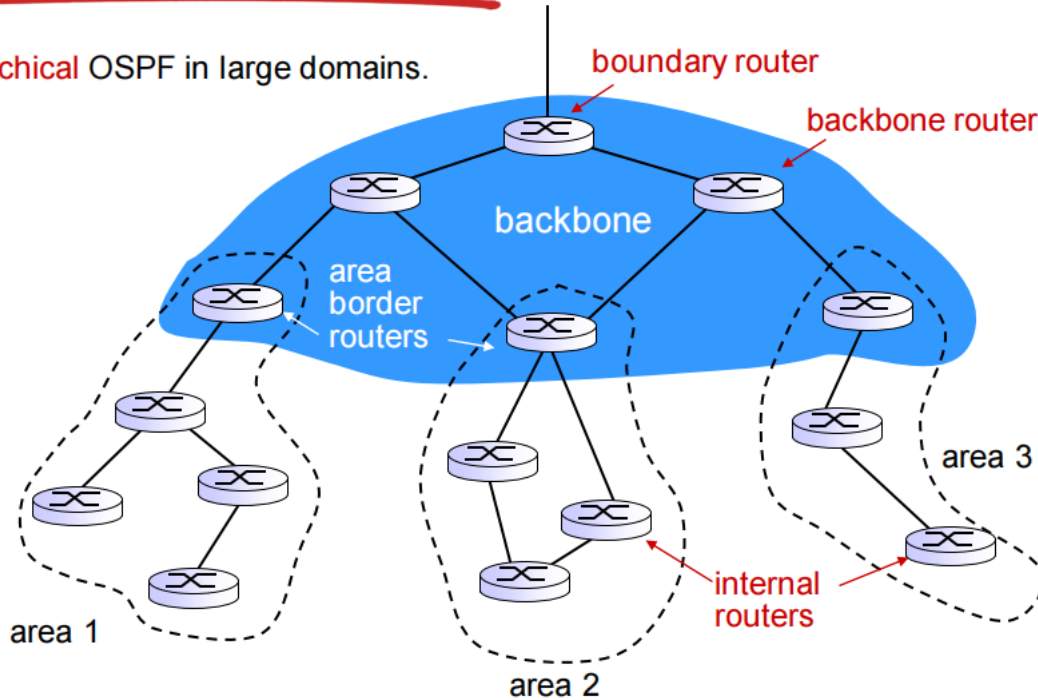
3. 区域边界路由器 (Area Border Routers, ABRs)：

每个区域至少有一个区域边界路由器，它连接骨干区域 (Area 0) 和其他区域。

ABRs 负责路由区域内部与外部的流量，起到桥梁作用。

Hierarchical OSPF

hierarchical OSPF in large domains.



Internet inter-AS routing: BGP

- ❖ **BGP (Border Gateway Protocol):** it is the standard for inter-AS routing in Internet.
 - “glue that holds the Internet together”
- ❖ **BGP Basic:**
 - One or more routers in each AS use BGP to communication with other such routers in other ASs to exchange routing information.
 - These routers are called as BGP peers.
 - Obtain subnet reachability information from neighboring ASs
 - Propagate the reachability information to all routers within the AS.
 - Determine “good” routes to subnets based on reachability information and policy

Network Layer

1. BGP 是一种标准化协议，能够在不同自治系统（AS，Autonomous System）之间交换路由信息。

BGP 的基本工作原理

1. BGP Peer (BGP 对等) :

- 每个 AS 内有一个或多个路由器运行 BGP。
- 这些路由器通过建立 **BGP 对等连接** (peer connection) , 与其他 AS 的路由器交换路由信息。

2. 子网可达性信息:

- BGP 获取相邻 AS 提供的子网可达性信息 (如某个子网的路径) 。
- 它将这些信息传递到本 AS 内的所有路由器。

3. 路径选择:

- 根据收集到的子网可达性信息和政策规则 (如 AS-Path 属性) , 选择到子网的“最佳路径”。

核心功能

- **路由信息传播**: BGP 广播子网的可达性信息, 确保所有路由器知道如何到达目标子网。
- **政策驱动**: 与纯粹的“最短路径”不同, BGP 更注重遵循网络运营商定义的路由策略, 比如避免某些路径或优先某些路径。

how Internet has made it possible to scale to millions of users.

- ❖ Routers are aggregated into autonomous systems (ASs). Within an AS, all routers run the same intra-AS routing protocol.
- ❖ Special gateway routers in the various ASs run the inter-autonomous system routing protocol that determines the routing paths among the ASs.
- ❖ The problem of scale is solved since an intra-AS router need only know about routers within its AS and the gateway router(s) in its AS.

互联网如何通过自治系统 (AS) 实现大规模扩展, 从而支持数百万用户的通信?

1. 路由器聚合到自治系统 (AS) 中

- 互联网中的路由器根据组织或网络域划分为自治系统 (AS, Autonomous System) 。每个 AS 是一个独立管理的网络, 可能属于一个企业、学校或互联网服务提供商。
- 在同一个 AS 内, 所有路由器运行相同的**内部路由协议** (IGP, Interior Gateway Protocol) , 例如 OSPF 或 RIP。这种统一的协议简化了 AS 内部的路由管理。

2. 网关路由器 (Gateway Router) 的作用

- 在一个 AS 中，某些路由器被指定为网关路由器 (Gateway Router) ，用于连接到其他 AS。
- 网关路由器运行外部路由协议 (EGP, Exterior Gateway Protocol) ，例如 BGP (Border Gateway Protocol) 。
- BGP 负责决定不同 AS 之间的路由路径，通过交换路由信息确保数据包可以在多个 AS 中正确传递。

3. 解决规模问题

- 通过这种架构设计，路由器只需要了解：
 - 本 AS 内部的路由情况。
 - 本 AS 中的网关路由器位置。
 - 这种分层设计减少了路由器需要维护的全局信息量，从而解决了规模问题，支持互联网扩展到数百万用户。
-

六、Broadcast and multicast routing

Three types of communications

□ Unicast

- Single source, single receiver

□ Broadcast

- Send same packet to all receivers in a LAN, subnet, or in an organization.

□ Multicast

- Send some packet to multiple receivers
- Applications: bulk data transfer (e.g., software upgrade from the developer to users needing the upgrade), remote education (the transfer of the audio, video, and text of a live lecture to a set of students) , teleconference, etc.

1.1.1. 单播 (Unicast)

一对一，数据从一个源发送到一个接收方。

适用于需要专属连接的场景，如网络请求、电子邮件发送。

1.1.2. 广播 (Broadcast)

广播是一对全的通信方式，数据从一个源发送到所有网络中的接收方

特点：

- 数据包被发送到网络内的所有设备，无论是否需要。
- 广播的范围通常限制在局域网（LAN）或子网。

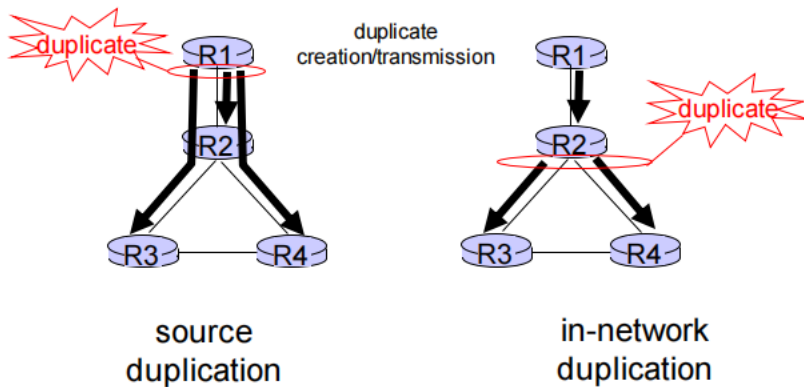
1.1.3. 组播 (Multicast)

组播是一对多的通信方式，数据从一个源发送到多个特定接收方

相较于广播，更高效，避免浪费网络资源。

- 应用场景：
 - 远程教育：实时传输视频和音频到多个学生。
 - 视频会议：在多个接收点传递流媒体数据。
 - 软件分发：从开发者传递更新给需要升级的用户。
-

Broadcast / Multicast



Source duplication (N-way unicast): Given N destination nodes, the source node simply makes N copies of the packet, addresses each copy to a different destination, and then transmits the N copies to the N destinations using unicast routing.

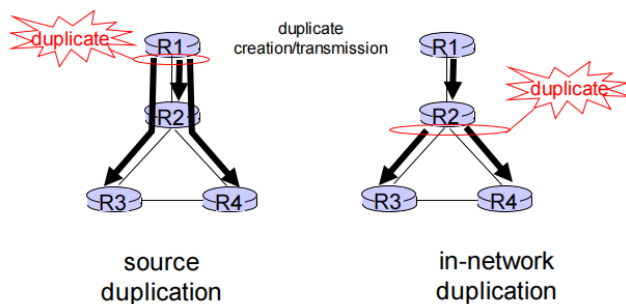
- Simple (advantage)
- Inefficient (disadvantage)
- Source may not know the addresses of all receivers (disadvantage)

Network Layer 4-111

源复制 (N-way Unicast)

源节点会创建数据包的多个副本，并通过单播方式将每个副本分别发送到各个目标。

1. 优点：实现简单，源节点只需按照单播路由规则发送数据包。
2. 缺点：效率低下：源节点需要处理和发送多份数据，占用大量的网络带宽；未知地址问题：源节点可能并不知道所有接收者的地址。



In-network duplication: to make copy of packet at the network nodes, rather than at the source node. It is more efficient than source duplication.

网络内复制 (In-Network Duplication)

在网络中的中间路由器或节点完成复制操作

- 更高效：避免了源节点发送多份数据包，减轻了源节点的负担，也更有效利用了网络带宽。
- 节省资源：减少了重复传输带来的冗余流量。

总结

- **源复制**实现简单但效率较低，适用于小规模网络或目标数较少的场景。
- **网络内复制**更高效，适合需要在大规模网络中向多个目标节点传输数据的场景（如组播）。